arXiv:2605.18692v2 [cs.AI] 27 May 2026

# Democratizing Large-Scale Re-Optimization with LLM-Guided Model Patches

Tinghan Ye,[1,*] Arnaud Deza,[1] Ved Mohan,[1] El Mehdi Er Raqabi,[1,2] Pascal Van Hentenryck[1]

[1] *H. Milton Stewart School of Industrial and Systems Engineering, Georgia Institute of Technology, Atlanta, USA*

[2] *Department of Operations and Decision Systems, Université Laval, Québec, Canada*

[*] *Corresponding Author: joe.ye@gatech.edu*

**Abstract**

Optimization models developed by operations research (OR) experts are often deployed as decision-support systems in industrial settings. However, real-world environments are dynamic, with evolving business rules, previously overlooked constraints, and unforeseen perturbations. In such contexts, end users (e.g., operators or planners) should ideally re-optimize models to recover feasible and implementable solutions, often without access to the original model developers. In general, however, they often work with expert modelers to apply necessary changes, inducing delays and preventing real-time applications.

This paper introduces an agentic re-optimization framework in which a large language model (LLM) acts as an OR expert, dynamically supporting end users through natural-language interaction. The LLM translates user prompts into structured updates of the underlying optimization model, selects suitable re-optimization techniques from an optimization toolbox, and solves the resulting instance to return implementable solutions. The toolbox leverages primal information, including historical solutions, valid inequalities, solver configurations, and metaheuristics, to accelerate re-optimization while preserving solution quality. The proposed framework enables interactive and continuous adaptation of deployed optimization models, reducing dependence on OR experts, and improving the sustainability of decision-support systems.

Extensive experiments on two complementary large-scale real-world case studies demonstrate the effectiveness and scalability of the proposed framework. The first considers online supply chain re-optimization, where solutions must be generated rapidly while remaining close to the deployed plan, whereas the second focuses on offline university exam scheduling, where solution quality is prioritized over runtime. Together, these settings highlight the ability of the framework to support both real-time operational recovery and high-quality planning. Results show that the toolbox-driven architecture significantly improves computational efficiency through primal-based and solver-aware re-optimization techniques, while the structured patch-based updates improve interpretability and traceability of model modifications.

*Keywords*: Large-scale optimization, re-optimization, large language models, mixed-integer programming, primal heuristics, agentic AI, decision support systems

# 1 Introduction

Modern business environments are increasingly complex and dynamic, challenging the traditional lifecycle of optimization models in operations and management (Er Raqabi et al., 2025). Classical operations research (OR) practice relies on expert modelers to translate real-world problems into mathematical formulations, calibrate them to data, and deliver executable tools to organizations. Once deployed, however, these models often lose relevance as operational requirements and data landscapes evolve. Each adaptation typically requires renewed expert intervention, an unsustainable process that limits the long-term impact of even the most sophisticated optimization models. In practice, even small operational adjustments can require nontrivial reformulation efforts, making continuous human maintenance both costly and error-prone in large-scale systems.

Given the dynamism and complexity, businesses frequently face rapidly changing business rules, evolving requirements, and unexpected perturbations. This impacts the developed OR models in various ways, including localized updates to constraints, variables, and model parameters required to maintain operational feasibility and decision quality. As a result, users must re-optimize models continuously to maintain solution quality and operational feasibility. Current tools, however, rarely equip users to adjust models autonomously. This creates an important organizational bottleneck, where the ability to adapt deployed optimization systems becomes limited by the availability of specialized modeling expertise and the latency associated with repeated model maintenance cycles. Importantly, this re-optimization task is not merely a re-solving exercise, but a structured model-editing problem that requires understanding how local changes propagate through a tightly coupled optimization system.

Although OR models are often sophisticated, they are inherently generic and do not account for case-specific adaptations. When deployed, end users encounter new or previously overlooked business rules that require immediate adjustments (see e.g., Er Raqabi et al. (2025)). The conventional workflow, returning to the modeler for each update, creates delays and reduces practical value. Although organizations can manually implement common update pathways, maintaining exhaustive adaptation logic becomes increasingly difficult as optimization systems scale, evolve, and interact with diverse operational scenarios. Addressing this gap calls for frameworks that enable users to modify and dynamically re-optimize models in line with evolving operational rules. This is

nowadays possible with the recent advances in large language models (LLMs). The LLMs offer new opportunities for interactive and natural-language-based problem-solving. While LLMs have been applied mainly to code generation or decision support, their potential to sustain and evolve formal re-optimization models in large-scale real-world settings remains largely unexplored (Li et al., 2023; Kong et al., 2025; Simchi-Levi et al., 2025a). However, a key limitation of existing approaches is that they typically treat re-optimization as either (i) pure code generation or (ii) black-box repair, without explicitly reasoning over the underlying mathematical structure of the model, the feasibility implications of edits, or the validation and traceability requirements necessary for deployment in operational environments, particularly in *large-scale* settings where even minor modifications can have significant computational and structural impact. This gap is fundamental since OR experts are still required because understanding *what to change* in a large-scale mixed-integer program (MIP) is often more difficult than solving it, exposing a key *interpretability* bottleneck. Even experienced OR practitioners must trace constraint interactions, variable dependencies, and feasibility boundaries under tight time constraints, a task that becomes increasingly impractical at an industrial scale due to model size and structural coupling.

This paper proposes an interactive agentic framework, **ReOpt-LLM**, in which an LLM bridges the OR model and the end user, serving as an interactive orchestration layer for model adaptation and re-optimization. Once the core model is provided by the OR expert, the LLM manages case-specific adaptations by translating user inputs into model updates and coordinating re-optimization. This setup allows users to interact dynamically with the model, updating parameters, variables, and constraints in real time without necessarily having OR expertise. This is inspired by and aligns with practice, where many business rules are highly customized or may not have been considered during model development by the OR expert. Consequently, the delivered OR tool must allow continuous operation and adaptation to emerging updates without constant expert intervention. **ReOpt-LLM** leverages the complementary strengths of OR experts and LLMs. Each prompt is translated into changes in model parameters, variables, or constraints, and the LLM serves as an interactive intermediary between the user and the OR model, enabling dynamic re-optimization that recovers and maintains decision quality. One of the main strengths of the proposed framework is its interaction with a re-optimization *toolbox*, which contains primal information and various

OR techniques, including historical solutions, solver configurations, valid inequalities, and domain-specific heuristics. This toolbox supports quick re-optimization following a prompt by the user. Importantly, each modification remains explicit, traceable, and verifiable through structured model patches and validation steps, allowing users to inspect how requests are translated into optimization updates and how the resulting solutions satisfy the intended operational changes. A central design principle is that the LLM does not replace optimization expertise, but instead operationalizes it by orchestrating a structured set of solver-aware actions, thereby reducing the need for direct human modeling intervention while preserving mathematical rigor.

Throughout **ReOpt-LLM**, the aim is to advance the sustainability of decision-support systems in real-world settings, reducing dependence on continuous expert intervention while ensuring long-term alignment with changing operational realities. This also contributes to the emerging literature on LLM-assisted decision making, demonstrating how language-based intelligence can bridge the gap between model formulation and practical implementation. By embedding re-optimization capabilities directly within an LLM-guided orchestration framework, the proposed approach transforms deployed optimization models from static analytical artifacts into continuously adaptive decision-support systems. A key additional insight is that the effectiveness of such systems is not uniform across problem scales. While simpler or well-conditioned instances can often be handled with minimal intervention, large-scale structured MIPs require careful coordination among model editing, heuristic guidance, and solver configuration, motivating the proposed toolbox-driven design.

The contributions of this paper are the following:

1. **Agentic Re-optimization Framework.** A dynamic and interactive re-optimization framework connecting the OR model, the re-optimization toolbox, and the end user, enabling continuous integration of user input and adaptation to diverse operational contexts.
2. **OR Perspective.** LLM-assisted re-optimization by translating user queries into mathematical model updates. Once the mathematical model is updated, the LLM selects *suitable* re-optimization modules from the toolbox and solves the resulting instance using these modules and an optimization solver. The toolbox is a key driver of scalability, enabling efficient large-scale re-optimization through reusable primal information, warm starts, solver configurations, valid inequalities, and customized heuristics. This design allows the framework to

handle large-scale instances under strict time budgets while maintaining solution quality.

3. **LLM Perspective.** Methods to prompt, guide, and structure the LLM for reliable re-optimization, including structured patch-based model edits, toolbox-aware decision making, and failure analysis under model-edit ambiguity. These patch operations improve interpretability and deployment reliability by making every modification to the optimization model explicit, traceable, verifiable, and reviewable by the end user, rather than implicit or black-box.
4. **Applications.** Extensive experiments on two real-world large-scale problems highlight the ability of the framework to re-optimize reliably and return satisfactory solutions within reasonable execution time. The two case studies are complementary. The OCP Group case represents an *online re-optimization* setting, where perturbations occur during the execution of a supply chain, decisions must be updated under strict time constraints, and new solutions must remain as close as possible to the already deployed plan, making both computational speed and solution stability critical. In contrast, the Cornell exam scheduling case corresponds to an *offline re-optimization* setting, where the schedule is iteratively refined with the university registrar before publication, and feasibility and solution quality are prioritized over runtime. Together, these two settings demonstrate the robustness of the proposed framework across both real-time operational decision-making and pre-deployment planning environments.

The remainder of the paper is organized as follows. Section 2 reviews relevant literature. Section 3 states the problem setting. Section 4 presents the **ReOpt-LLM** framework. Section 5 provides the mathematical formalization and the implementation. Section 6 illustrates a toy example. Section 7 describes the shared evaluation protocol used in the case studies. Sections 8 and 9 present two large-scale real-world applications. Section 10 concludes with a discussion of broader implications, challenges, and directions for future research.

## 2 Literature Review

This section reviews prior work relevant to this research. It first examines the emerging role of LLMs in optimization, then discusses re-optimization techniques in OR, and finally positions this work relative to these streams.

## 2.1 LLMs for Optimization

The application of LLMs to the field of optimization represents a nascent yet rapidly advancing area of research, spanning model formulation, solution support, human-in-the-loop decision making, and optimization education. Several recent position papers, surveys, and case studies have provided foundational perspectives on the emerging role of LLMs in various optimization contexts and their potential to democratize solution modeling (Li et al., 2023; Simchi-Levi et al., 2025a,b; Chen et al., 2025b; Xiao et al., 2025). Research in this area explores several distinct roles for LLMs across the optimization process. This distinction is useful for positioning this work, which focuses on the post-deployment stage where an already validated optimization model must be continuously adapted and re-optimized under evolving operational conditions.

A primary application, NL4OPT (natural language for optimization), focuses on the modeling and formulation phase. This involves converting unstructured natural language descriptions into formal optimization formulations (Ramamonjison et al., 2023). This task is typically approached using prompt-based techniques (Xiao et al., 2024; AhmadiTeshnizi et al., 2024; Kong et al., 2025; Liang et al., 2025; Zhang et al., 2026) or learning-based methods (Shu et al., 2025; Huang et al., 2025; Zhou et al., 2025). Related work also investigates using LLMs for other pre-solving tasks, such as configuring solver algorithms (Lawless et al., 2025), and post-processing tasks, like filtering a pool of Pareto-optimal solutions through a multi-objective lens (Jovine et al., 2026). Closely related, OptiChat (Chen et al., 2025a) provides a conversational interface to an already-built optimization model for interpretation, infeasibility diagnosis, sensitivity analysis, and transient what-if evaluation. Drossman et al. (2026) study a related conversational setting in which an agent makes bounded edits to an already-deployed formulation in response to stakeholder preferences, such as fixing a variable value or imposing an objective bound. In that setting, the underlying decision space remains unchanged across the conversation. These works primarily address the construction of optimization models from initial problem descriptions, the explanation of an existing one, or the elicitation of stakeholder preferences over one. This paper also starts from a validated deployed model, but it targets operational changes that arise after deployment. Here, user requests introduce new operational adaptations or previously uncaptured requirements that may change the feasible region. The agent translates those requests into structural updates of the model's parameters, vari-

ables, constraints, or objective, and then applies solver-aware re-optimization techniques instead of solving the revised model from scratch.

Beyond problem formulation, another research thrust integrates LLMs directly into the solution process. This ranges from positioning the LLM as a direct, formulation-free optimizer (Yang et al., 2024) to embedding it as an active component within traditional algorithms. Spurred by advances in LLM-based code generation (Novikov et al., 2025), these more integrated approaches use LLMs to dynamically generate or select mathematical heuristics (Romera-Paredes et al., 2024), to act as sophisticated crossover or mutation operators in evolutionary algorithms (Ye et al., 2024), or to guide the selection policy in Monte Carlo Tree Search (Zheng et al., 2025). ReOpt-LLM differs from this stream in that the LLM is not used as the optimizer itself or as an unconstrained heuristic generator; rather, it acts as an OR-aware orchestration layer that coordinates structured model adaptation, validation, and solver-aware re-optimization techniques.

LLMs are also emerging as a novel mechanism for managing the human-in-the-loop aspect of optimization, particularly for preference elicitation. Their capacity for natural language interpretation enables zero-shot preference modeling, allowing a decision-maker's objectives to be inferred directly from verbal descriptions rather than through explicit pairwise queries. To date, this research has primarily employed LLMs as auxiliary components to guide interactive questioning (Lawless et al., 2024; Austin et al., 2024), extract latent preferences from unstructured text (Bang and Song, 2025), or serve as simulators of user behavior (Okeukwu-Ogbonnaya et al., 2025; Zhang et al., 2025). These studies use language primarily to elicit, infer, or simulate preferences over decisions, whereas the focus in this paper is on operational updates that alter the feasible region, objective, or model data and therefore require re-solving the modified optimization problem.

Finally, complementing these application-oriented efforts, a separate stream of work evaluates the foundational knowledge of LLMs on core optimization principles. For instance, studies have assessed their understanding of primal-dual theory, which is crucial for ensuring reliable modeling and for their potential use in education (Klamkin et al., 2025).

## 2.2 Re-optimization Techniques

Re-optimization plays a central role in maintaining robustness and adaptability in environments subject to continuous or unexpected change. It is commonly divided into two broad categories. *Major re-optimization* refers to infrequent but substantial redesigns triggered by significant disruptions. These are typically strategic or tactical in nature and are often resolved from scratch using exact optimization methods. In contrast, *minor re-optimization* deals with routine or operational disturbances that occur frequently, sometimes in real time. In this setting, solutions are not rebuilt entirely. Instead, previously computed solutions are adapted using heuristics or approximation schemes that exploit existing structure.

From the perspective of large-scale or structural re-optimization, Chen and Miller-Hooks (2012) introduce a stochastic mixed-integer formulation to assess system resilience and to determine recovery actions after disruptions. Similarly, Bruno et al. (2021) study the redesign of a postal collection network in Italy, where the decision problem involves selecting urban postboxes to deactivate. They formulate the problem as a mixed-integer program and solve it using a two-stage exact approach.

Minor re-optimization, on the other hand, focuses on fast adaptation to localized changes. For example, D'Ariano et al. (2010) consider adjustments to train running profiles using a graph-based representation combined with constructive heuristics that preserve feasibility while modifying existing schedules. Archetti et al. (2013) study variations of the Rural Postman Problem and propose heuristic strategies to update solutions when edges are added or removed. Within a more general combinatorial framework, Schieber et al. (2018) formalize combinatorial re-optimization problems and develop methods that explicitly control transition costs between solutions.

In operational settings, Dong et al. (2018) investigate maritime inventory routing under uncertainty using a rolling-horizon scheme solved with a commercial MIP solver. Evolutionary approaches are examined by Doerr et al. (2019), who show that standard population-based methods can be inefficient in dynamic environments and propose mechanisms based on diversity preservation to improve adaptability. In workforce scheduling, Hassani et al. (2020) and Hasani et al. (2021) design fast heuristic procedures that generate near-Pareto solutions by explicitly accounting for cost and schedule deviations, often embedding neighborhood structures within integer programming formu-

lations. More recently, Er Raqabi et al. (2024) propose a general framework for resilience-driven re-optimization, where disruptions and recovery actions are jointly modeled. Their approach leverages primal solutions through fixing strategies, warm starts, valid inequalities, and machine learning components, and is validated on large-scale industrial instances.

Recent work relying on artificial intelligence (AI) highlights the challenge of deciding when to re-solve optimization models in real time. The proximal policy optimization (PPO) framework addresses this by learning optimal re-solving times, balancing solution quality with computational cost in dynamic MIP settings (Ai et al., 2025).

### 2.3 Paper Positioning

This paper lies at the intersection of OR, AI, and business analytics, addressing the challenge of sustainable model-driven decision-support systems. It proposes a framework that assumes that *OR experts developed and validated a mathematical optimization model, which is then deployed to end users*. In practice, the main difficulty is no longer model formulation or optimization, but the continuous and correct adaptation of a large, tightly coupled MIP under evolving operational conditions, where even small edits can have non-local and non-obvious feasibility and optimality implications. This makes re-optimization fundamentally a reasoning problem over model structure rather than a mere computational task. The LLM agent (i) interprets user queries (in natural language) containing operational perturbations, evolving operational requirements, and other structural updates, (ii) translates them into corresponding model modifications, (iii) picks suitable generic and/or domain-specific re-optimization techniques, (iv) re-optimizes the model under new or perturbed conditions by calling an optimization solver enhanced with the re-optimization techniques, and (v) delivers actionable insights in an *intuitive* form, i.e., a form that the end user can understand and implement. A key distinction from prior work is that these steps are not treated as independent tasks, but as a tightly coupled closed-loop decision system in which model editing, solver configuration, and primal enhancement interact. Unlike prior LLM-based optimization approaches that focus primarily on model generation, code synthesis, or black-box solution repair, this framework explicitly operates on structured, auditable, and reviewable model edits supported by explicit validation and traceability mechanisms, and integrates solver-aware re-optimization decisions grounded in OR methodol-

ogy. This is crucial because naive code-level modifications are insufficient to guarantee feasibility or meaningful control over combinatorial structures in large-scale MIPs. By embedding re-optimization capabilities within the LLM, **ReOpt-LLM** transforms traditional optimization tools from static artifacts into continuously adaptive and collaborative decision-support systems that evolve through human–AI collaboration. Importantly, the LLM is not positioned as a solver or optimizer replacement, but as an *OR-aware orchestration layer* that externalizes expert reasoning (model diagnosis, modification selection, and solver guidance) in a way that is both scalable and interpretable. This allows domain users to interact with complex optimization systems without requiring deep expertise in mathematical modeling, while preserving deployment reliability through solver validation, structured feedback, and explicit human review of model modifications. Overall, the framework views re-optimization as a structured decision-understanding problem, where language, model structure, and primal information are jointly leveraged to sustain feasibility, operational responsiveness, and decision quality under continuous change. To the best of the authors' knowledge, no prior work unifies structured model editing, solver-aware re-optimization, and primal-toolbox selection within a closed-loop LLM-driven framework for large-scale re-optimization.

## 3 Problem Description

Consider a generic optimization setting in which an OR model is developed, validated, and deployed to support decision-making in a dynamic and complex environment. The deployed model constitutes a well-defined computational object that encodes the operational logic of the system. Let $x$ denote the vector of decision variables, partitioned into continuous and integer components, and consider the following MIP formulation:

$$\min_{x} \left\{ c^\top x \;\middle|\; Ax \leq b,\ x \in \mathbb{R}^n \times \mathbb{Z}^m \right\}, \tag{3.1}$$

where $c$ is the cost vector, $A$ is the constraint matrix, and $b$ is the right-hand side vector. This formulation captures a wide range of decision-support applications, including production planning, scheduling, transportation, and resource allocation. This work assumes that the underlying problem structure remains a minimization problem of the form (3.1), while its components may evolve over time. Consequently, the MIP is not static, but a parameterized and structured model whose elements

may be modified as the environment changes.

In practice, such models are often large-scale and highly structured, involving thousands to millions of variables and constraints. They are typically designed and calibrated by OR experts in collaboration with domain specialists, and then deployed as decision-support tools within organizations. Once deployed, these models are used repeatedly to generate tactical or operational plans under varying data inputs. However, real-world environments are inherently dynamic, and the conditions under which the model was originally designed may no longer hold. Over time, perturbations arise, operational requirements evolve, and previously uncaptured constraints or structural adjustments emerge. These changes may affect any component of the model, including the cost vector $c$, the decision variables $x$, the constraint matrix $A$, or the right-hand side vector $b$. For example, new constraints may need to be introduced, existing constraints modified or removed, variable domains restricted or expanded, or parameter values updated. Crucially, such modifications are often described locally in natural language, yet they induce global and highly non-trivial effects on feasibility and optimality. In practice, these updates are often initiated by planners, operators, or other domain users who understand the operational context but lack expertise in mathematical optimization. Consequently, effective re-optimization requires not only computational efficiency but also interactive mechanisms that allow users to express intended operational changes, inspect the resulting model modifications, and validate that the revised solution aligns with operational objectives and constraints.

As a result, the deployed model must be continuously re-optimized to remain relevant and effective. Importantly, this re-optimization process goes beyond simply re-solving the model with updated data. It requires modifying the mathematical structure of the optimization problem itself. Ideally, these modifications should remain explicit and inspectable, allowing users to understand how localized operational requests alter the underlying optimization model. From a computational perspective, this is challenging because even small changes in the model components can significantly alter the feasible region and the solution space, especially for large-scale models. *From a practical perspective, this is difficult because end users typically lack the expertise required to manipulate mathematical models directly, and repeated reliance on OR experts is costly and slow.* Although some recurring updates can be manually encoded within decision-support software, exhaustively

anticipating and maintaining all possible operational adaptations becomes increasingly difficult as optimization systems scale, evolve, and interact with diverse operational scenarios. Moreover, these updates must be performed quickly to support real-time or near-real-time decision-making, creating a tension among model expressiveness, solution quality, and computational efficiency. Delays in adapting deployed optimization systems can significantly reduce operational responsiveness and diminish the practical value of model-driven decision-support tools. A central difficulty is that identifying what structural edits preserve feasibility while improving or maintaining solution quality is often strictly more difficult than solving the resulting optimization problem itself. Therefore, even in the absence of language models, enabling systematic, reliable, and fast re-optimization of evolving large-scale MIP models is a challenging and important problem.

This setting motivates the need for a framework that can (i) interpret user-specified changes expressed in natural language, (ii) translate these changes into precise, reviewable, and verifiable updates of the mathematical model (3.1), and (iii) efficiently re-optimize the resulting instance while preserving solution quality and implementability. The next section presents an LLM-assisted agentic framework designed to address this challenge.

## 4 Agentic Re-optimization Framework

**ReOpt-LLM**, the proposed LLM-assisted collaborative re-optimization framework, is illustrated in Figure 1; it integrates the end user, the optimization model, and the optimizer through a language-based interface. The optimization model is assumed to have been developed and validated by OR expert(s) in collaboration with the end user's organization (*Step 0*). As the business environment evolves, e.g., due to perturbations or changing operational requirements, the existing model may no longer produce valid solutions, necessitating re-optimization (*Step 1*). The user communicates these emerging issues via natural-language prompts to the LLM (*Step 2*), which translates them into corresponding changes in the model parameters and variables (*Step 3*). The updated model is then instantiated to generate new problem instances (*Step 4*), which are solved by an optimizer to produce feasible and high-quality solutions (*Step 5*). The optimizer uses re-optimization techniques from the toolbox to enhance the solver. The re-optimized solution is returned to the user, forming a continuous feedback loop that combines human expertise, AI, and optimization for real-time,

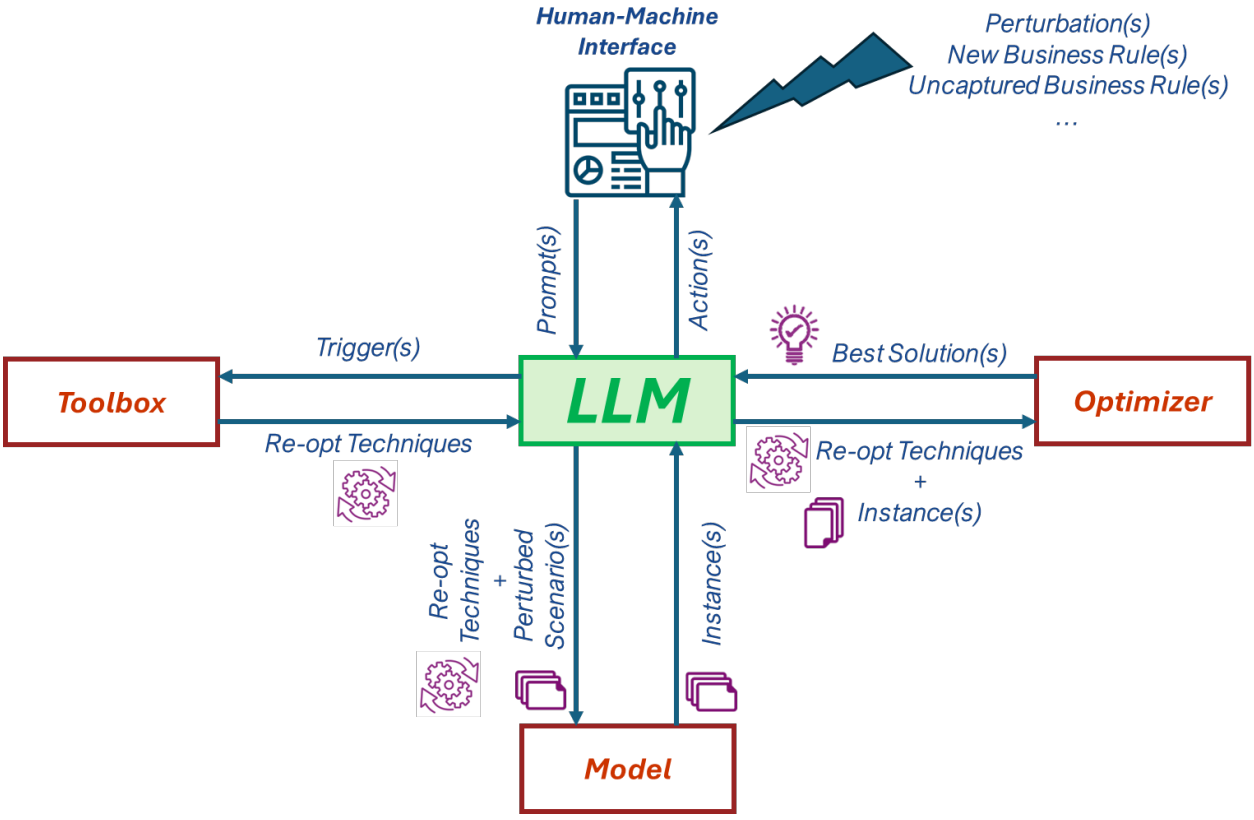



Figure 1: **ReOpt-LLM** Framework

adaptive decision support. The rest of the section describes **ReOpt-LLM** in detail, presenting the step-by-step operation, the LLM trigger mechanism, workflow, and role, the metrics used to evaluate LLM performance, and the speed and optimization techniques.

### 4.1 Step-by-step Description

This section describes the seven steps involved in the **ReOpt-LLM** framework.

**Step 0 – Model Validation and Delivery.** The initial optimization model is developed by OR expert(s) and iteratively refined in collaboration with the end user's organization. Through repeated validation and testing, the model is calibrated to capture the company's operational logic, business rules, and performance objectives. Once finalized, the model is deployed within the organization for the end user, who can leverage it to optimize operations and, when needed, re-optimize as long as the underlying assumptions remain valid. This step ensures that the model reflects both technical rigor and practical relevance. In the absence of perturbations, the end user uses the model in the

normal loop, which starts from the *Human-Machine Interface* and goes to the *Model* through the dashed arrow in Figure 1.

**Step 1 – Identification of Changes.** Over time, the operational environment may change, introducing perturbations, new business rules, and/or previously uncaptured requirements. These changes can occur at any moment throughout the planning horizon, whether in real time, daily, or weekly, reflecting the continuous evolution of operations. *ReOpt-LLM is designed to incorporate all such emerging changes dynamically, ensuring continuous model adaptability.* When the existing model can no longer produce feasible or optimal solutions under these conditions, the end user recognizes the need to update and re-optimize, triggering the collaborative re-optimization process.

**Step 2 – User Prompting via LLM.** The end user communicates emerging operational challenges to the LLM through a natural-language interface. The prompt may describe new rules, exceptions, or situational changes in the business environment. By allowing the user to express requirements in natural language, the system reduces the barrier to model adaptation and encourages iterative interaction between human expertise and computational reasoning.

**Step 3 – Translation of Prompts into Model Updates.** The LLM interprets the user's prompts and translates them into precise modifications of the optimization model. Specifically, the LLM identifies the effects on the model components, including the cost vector, decision variables, right-hand side vector, and constraint matrix, while assuming that the underlying problem structure (e.g., a minimization problem) remains unchanged. The LLM thus produces an updated scenario that captures the operational impact of newly identified rules or perturbations. The resulting modifications remain explicit and inspectable, allowing the end user to review how the operational request alters the optimization model before re-optimization is performed.

**Step 4 – Toolbox Selection.** Based on the interpreted changes, the LLM selects appropriate generic and domain-specific re-optimization techniques from the toolbox. This step operationalizes OR expertise within the collaborative re-optimization workflow by mapping structural model modifications to suitable primal and algorithmic strategies. Depending on the nature of the perturbation, the LLM may activate warm-start mechanisms, exploit historical primal solutions, introduce valid inequalities, adjust solver parameters, or invoke specialized heuristics. These selections explicitly condition both the instance construction and the downstream optimization process, ensuring that

re-optimization is informed by problem structure rather than treated as a black-box solve.

**Step 5 – Instance Generation.** Using the modified model, new problem instances are generated that reflect the updated operational environment. These instances formalize the re-optimization task, ensuring that all relevant changes are incorporated into a solvable mathematical representation. This step bridges the translation performed by the LLM and the computational optimization process.

**Step 6 – Optimization and Solution Feedback.** The optimizer solves the updated instances, producing feasible and high-quality solutions under the revised conditions. The results are communicated, in the form of actionable decisions, back to the user, who can evaluate and implement them. The framework can additionally provide structured feedback describing the applied model updates, activated re-optimization strategies, and validation outcomes, helping users assess whether the generated solution aligns with the intended operational adaptation. This creates a continuous human-in-the-loop feedback process in which human expertise, AI, and optimization interact dynamically, supporting adaptive and sustainable decision-making in evolving operational contexts.

### 4.2 LLM Trigger Mechanism, Workflow, and Role

The re-optimization process is triggered whenever a user prompt implies a change in the underlying optimization problem. Any such prompt can be mapped to modifications in one or more of the core components that define the model: the cost vector, the decision variables, the constraint matrix, or the right-hand side vector. Notably, updates to these components allow the framework not only to adjust existing constraints but also to add new constraints or remove obsolete ones, enabling flexible adaptation of the model to evolving operational requirements. By framing the problem in this way, every user input becomes structured and actionable, providing a structured basis for the LLM to generate updates and for systematic evaluation of its performance. This structure also supports validation and human review before re-optimization, helping ensure that the generated modifications correctly reflect the intended operational adaptation.

To operationalize the framework, the LLM must first understand the *what*, i.e., the specific emerging issues, business rules, or perturbations described by the user. Once these impacts are identified, it determines how to update the scenario(s), translating the natural-language description into precise modifications of the cost vector, decision variables, right-hand side, and constraints.

With the updated scenario defined, the LLM constructs the corresponding updated optimization instance, ensuring that it accurately reflects the new operational conditions. This model is then submitted to the optimizer, which computes feasible solutions. Finally, the resulting solution is returned to the user, completing the cycle and enabling continuous, adaptive re-optimization.

### 4.3 Evaluation Metrics for the LLM

The performance of the LLM within the **ReOpt-LLM** framework can be evaluated along three complementary dimensions. The first dimension assesses the accuracy in identifying model components, i.e., whether the LLM correctly determines which elements, including the cost vector, decision variables, constraint matrix, or right-hand side, require updating in response to the user's prompt. The second dimension measures the correctness of the updates, i.e., whether the LLM modifies the identified components appropriately to reflect the intended operational changes. The third dimension evaluates the quality and usability of the resulting solutions, including feasibility, optimality, and consistency of the updated model; it also evaluates the relevance of the solutions for the end user, i.e., whether the prompt is correctly addressed. Together, these metrics provide a comprehensive assessment of the LLM's effectiveness in supporting real-time, adaptive decision-making.

### 4.4 Speed and Optimization Techniques

A key requirement of the **ReOpt-LLM** framework is that updates and solution generation should occur rapidly, enabling near real-time decision support even as operational conditions evolve. To achieve this, the toolbox can leverage a variety of OR techniques, including exact solvers, metaheuristics, accelerated exact methods, ML-guided approaches, and proxy models. By flexibly selecting or combining these methods, the framework ensures that high-quality solutions are produced efficiently, balancing computational speed with solution accuracy and implementability, while maintaining the responsiveness required for dynamic, continuous re-optimization in practical operational settings.

## 5 Mathematical Formalization and Framework Implementation

This section presents the mathematical formalization of the **ReOpt-LLM** framework and its implementation. Subsection 5.1 introduces the structured model representation, together with the

metadata used by the agents. Subsection 5.2 formalizes the dynamic setting and the structured events that summarize natural-language changes. Subsection 5.3 defines the patch language used to mutate the structured state. Subsection 5.4 describes the collaborative multi-module workflow, in which a single integrated patch planner jointly interprets the request and proposes candidate edits, a deterministic programmer normalizes them, a strategy selector picks a re-optimization strategy, and a validator solves the updated model under a validation-triggered retry loop. Subsection 5.5 combines these components into a closed-loop algorithm.

Consider a generic optimization problem of the form

$$\min_{x \in \mathcal{X}(\mathcal{M}, p)} \quad f(x; p), \tag{5.1}$$

where $x$ denotes the vector of decision variables, $\mathcal{M}$ denotes the structured model template, $p$ denotes the collection of model parameters (e.g., demands, processing times, exam sets, capacities, costs), and $\mathcal{X}(\mathcal{M}, p)$ is the feasible region defined by the constraints generated from $\mathcal{M}$ and parameterized by $p$. The feasible region is generated from a set of *decision variable families*, *constraint families*, and *objective components*.

## 5.1 Structured Model Representation

The optimization model has a structured representation of the form

$$\mathcal{M} = (\mathcal{V}, \mathcal{C}, \mathcal{O}),$$

where $\mathcal{V}$ is a collection of variable families, $\mathcal{C}$ is a collection of constraint families, and $\mathcal{O}$ is a set of named objective components. The parameter vector $p$ is shared across these collections, and $\mathcal{P}$ denotes the set of named parameter entries in $p$ (e.g., `supply`, `demand`, `costs`).

### 5.1.1 Decision variable families

Each variable family $v \in \mathcal{V}$ is associated with a finite index set $\mathcal{I}_v$ and a collection of decision variables

$$\{x_i\}_{i \in \mathcal{I}_v},$$

where all variables in the family share the same type and interpretation (e.g., flows on arcs, assignment variables, start times). Let $\text{type}(v) \in \{\text{binary}, \text{integer}, \text{continuous}\}$ denote the variable type for family $v$, and let $\ell_i$ and $u_i$ denote the lower and upper bounds for each $x_i$, which may depend on $p$.

### 5.1.2 Constraint families

Each constraint family $c \in \mathcal{C}$ is represented by: (1) **a finite index set** $\mathcal{I}_c$, (2) **a left-hand side specification** $g_c(x; p, i)$, and (3) **a right-hand side specification** $h_c(p, i)$. The instantiated constraints take the form

$$g_c(x; p, i) \;\leq\; h_c(p, i) \qquad \forall i \in \mathcal{I}_c,$$

where the sense $\leq$ can be replaced by $\geq$ or $=$. Different families correspond to different structural roles (e.g., capacity, balance, precedence, fairness).

### 5.1.3 Objective components

The objective function is decomposed into a finite collection of components indexed by $\mathcal{O}$:

$$f(x; p) \;=\; \sum_{k \in \mathcal{O}} w_k \, f_k(x; p),$$

where $f_k$ encodes a contribution such as transportation cost, tardiness, overtime, or fairness, and $w_k \in \mathbb{R}$ is a tunable weight.

### 5.1.4 Metadata for retrieval

Each retrievable object $\omega \in \mathcal{V} \cup \mathcal{C} \cup \mathcal{O} \cup \mathcal{P}$ carries (1) a short textual description $\text{desc}(\omega)$ and (2) a finite tag set $\text{tags}(\omega)$ (e.g., `capacity`, `routing`, `student`, `time-window`). These metadata fields do not affect the mathematical model but are exposed to the patch planner to help identify the components and parameter entries relevant to a given change request.

### 5.2 Dynamic Setting and Structured Events

Over time, new information arrives in two forms that describe parameter or structural modifications in $\mathcal{P}$, $\mathcal{V}$, $\mathcal{C}$, or $\mathcal{O}$:

- **Parameter updates**: the parameter vector $p$ is updated following a change (e.g., revised demands, machine calendars, updated travel times).
- **Natural-language rule updates**: a user or decision-maker expresses a change such as, *"Arc $(i, j)$ is unavailable today"*, *"Machine $m$ is down from 10:00–14:00"*, *"First-year students should not have evening exams"*, *"Product $q$ must be completed by tomorrow"*.

Let $\Delta_t$ denote the natural-language description of changes at time $t$. The mutable state is denoted by

$$\mathcal{Z}_t = (\mathcal{M}_t, \mathcal{P}_t, p_t),$$

where $\mathcal{M}_t$ collects variable, constraint, and objective-component families, $\mathcal{P}_t$ denotes the named parameter entries available at time $t$, and $p_t$ collects their current values. For compactness, define the state-dependent feasible region and objective as

$$\mathcal{X}(\mathcal{Z}_t) := \mathcal{X}(\mathcal{M}_t, p_t), \qquad F(x; \mathcal{Z}_t) := f(x; p_t).$$

The goal is to incrementally update $\mathcal{Z}_{t-1}$ to $\mathcal{Z}_t$ to incorporate the change $\Delta_t$, then solve

$$x_t^\star \in \arg\min_{x \in \mathcal{X}(\mathcal{Z}_t)} F(x; \mathcal{Z}_t),$$

efficiently and reliably, while preserving interpretability, correctness, and implementability. The framework first represents $\Delta_t$ as a structured event

$$E_t = \big(S_t, \iota_t, \sigma_t\big),$$

where $S_t$ is a description of the *affected entities*, encoded as subsets of index sets in the model (e.g., $S_t$ may refer to a subset of machines, arcs, students, or time slots, corresponding to subsets of some $\mathcal{I}_v$ or $\mathcal{I}_c$); $\iota_t \in \{\text{tighten}, \text{relax}, \text{forbid}, \text{prioritize}, \text{update}\}$ is an optional *semantic intention* retained

as auxiliary metadata; and $\sigma_t$ is a short textual summary of the requested edit. The structured event $E_t$ is an abstract representation of the change that the integrated planner conditions on $S_t$ and $\sigma_t$ when proposing model edits.

### 5.3 Patch Language

All updates to $\mathcal{Z}_t$ are expressed through a restricted *patch language*, i.e., a model-edit domain-specific language (DSL). Each patch is a tuple

$$\pi = (\texttt{op}, \texttt{target}, \texttt{scope}, \texttt{update}),$$

where `op` is an operation type, `target` refers to named parameter entries in $\mathcal{P}_t$ and their values in $p_t$ or to one or more elements of $\mathcal{V}_t$, $\mathcal{C}_t$, or $\mathcal{O}_t$, `scope` identifies relevant indices (e.g., particular machines, arcs, students, time slots), and `update` specifies the mathematical modification (e.g., new parameter values, new bounds, modified right-hand side, updated weight). After applying a patch $\pi$, the mutable state transitions to

$$\mathcal{Z}_t \;=\; \pi(\mathcal{Z}_{t-1}).$$

Multiple patches generated for the same change request can be grouped into a *candidate action set* that is applied as a unit.

The supported patch-operation vocabulary is summarized in Table 5.1. Operations on individual indices cover parameters, variable bounds, constraint right-hand sides, left-hand-side specifications, objective coefficients, and objective weights. Pattern-based operations support batch edits over many rows or variables selected by a structural pattern in LP-backed runtimes, which is useful when a single natural-language rule modifies a large family at once. Structural operations create or remove entire families.

The vocabulary is intentionally small. Most operational prompts are handled as data edits, such as changing a demand value, tightening a route bound, or adjusting an objective coefficient; these operations keep the algebraic structure fixed and only mutate existing entries. When a prompt states a relative change, such as "increase demand by 10," the corresponding patch may carry a `delta` rather than forcing the planner to compute the new absolute value. Pattern-based LP

operations are reserved for cases in which the runtime exposes named rows or variables but not a compact structured family. Structural operations are used only when the requested change adds or removes an entire modeling object, such as a new policy constraint family.

| Operation | Effect |
|---|---|
| `UPDATE_PARAMETER` | Replace or additively update a named parameter entry, optionally at a keyed index. |
| `UPDATE_BOUND` | Set the lower or upper bound of an existing variable-family member. |
| `UPDATE_CONSTRAINT_RHS` | Replace or additively update the right-hand side of an existing constraint-family row. |
| `UPDATE_CONSTRAINT_LHS` | Replace the left-hand-side specification $g_c(x; p_t, i)$ for an existing constraint family. |
| `UPDATE_OBJECTIVE_COEFF` | Replace or additively update an indexed coefficient inside objective component $f_k$. |
| `UPDATE_OBJECTIVE_WEIGHT` | Replace or additively update the weight $w_k$ of a named objective component. |
| `UPDATE_COEFFICIENT` | Modify nonzero LP matrix coefficients selected by variable and constraint-name patterns. |
| `FIX_VARIABLES_BY_PATTERN` | Set lower and upper bounds for LP variables selected by a name pattern and optional filters. |
| `UPDATE_CONSTRAINT_RHS_BY_PATTERN` | Set or scale right-hand sides for LP rows selected by a constraint-name pattern. |
| `ADD_VARIABLE_FAMILY` | Add a new decision-variable family, $\mathcal{V}_t \leftarrow \mathcal{V}_t \cup \{v'\}$. |
| `ADD_CONSTRAINT_FAMILY` | Add a new constraint family, $\mathcal{C}_t \leftarrow \mathcal{C}_t \cup \{c'\}$. |
| `REMOVE_CONSTRAINT_FAMILY` | Remove an existing constraint family, $\mathcal{C}_t \leftarrow \mathcal{C}_t \setminus \{c'\}$. |
| `ADD_OBJECTIVE_COMPONENT` | Add a new objective component, $\mathcal{O}_t \leftarrow \mathcal{O}_t \cup \{k'\}$. |

Table 5.1: Supported patch-operation vocabulary. Pattern-based operations apply to a regex-selected subset of LP rows or variables. Structural operations modify $\mathcal{V}$, $\mathcal{C}$, or $\mathcal{O}$.

## 5.4 Agentic Workflow and Retry Loop

The **ReOpt-LLM** framework processes each change $\Delta_t$ through three agent modules and one deterministic post-processor, illustrated in Figure 2. Agent 1, the integrated patch planner, jointly interprets the request and proposes candidate edits in a single LLM call. The deterministic programmer normalizes the result. Agent 2, the re-optimization strategy selector, picks a solve strategy from the toolbox. Agent 3, the validator and optimization engine, applies the edits and solves the updated model. A validation-triggered retry loop wraps the whole workflow.

**Agent 1: Patch Planner.** The patch planner takes the natural-language description $\Delta_t$ together with a representation of the current state $\mathcal{Z}_{t-1}$ that includes the metadata desc$(\cdot)$ and tags$(\cdot)$ for every model component and named parameter entry. In a single LLM call, the planner returns the

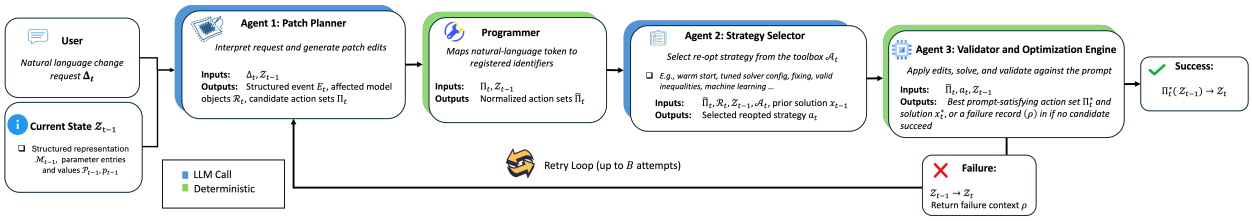



Figure 2: **Zoom on Framework.** A bounded repair loop processes the user request $\Delta_t$ through three agents: the Patch Planner generates candidate edits, the Strategy Selector chooses a re-optimization strategy from the toolbox, and the Validator + Optimization Engine applies the edits and solves. On validation failure, additional context $\rho$ is returned to Agent 1 (up to budget $B$). On success, the state advances to $\mathcal{Z}_t$, and the system handles the next request.

implemented structured-event fields corresponding to $E_t$, the relevant objects

$$\mathcal{R}_t \subseteq \mathcal{V}_{t-1} \cup \mathcal{C}_{t-1} \cup \mathcal{O}_{t-1} \cup \mathcal{P}_{t-1},$$

and a collection of candidate action sets

$$\Pi_t = \{\Pi_t^{(1)}, \Pi_t^{(2)}, \ldots\},$$

where each $\Pi_t^{(r)} = \{\pi_{r,1}, \pi_{r,2}, \ldots\}$ is an ordered group of patches intended to be applied together, with $r$ indexing candidate action sets. Combining interpretation and planning in one agent removes the extra interaction step between a separate classifier and a downstream patch generator: the planner conditions the proposed edit directly on $\Delta_t$, $\mathcal{R}_t$, and the metadata of the candidate target objects, which keeps the model's view of the request and the proposed action consistent. Working at the action-set level allows the planner to express edits that are only correct as a coordinated group, such as adding a new constraint family alongside an update to a related parameter.

**Programmer.** Each candidate action set is normalized by a deterministic, problem-specific routine before it reaches the solver. Let $\widetilde{\Pi}_t = \texttt{Normalize}(\Pi_t, \mathcal{Z}_{t-1})$ denote the resulting collection of normalized action sets. The programmer canonicalizes entity labels (e.g., maps the natural-language token `"Plant 1"` to the registered identifier `P1`), reconciles index types between the planner's output and the model's internal index sets (e.g., promotes the list `[P2, C2]` to the tuple `(P2, C2)` expected by the `flows` family), and, when needed, rewrites operations into a more efficient or more numerically stable form (e.g., replacing an `UPDATE_CONSTRAINT_RHS` on the supply constraint with

an equivalent `UPDATE_PARAMETER` on the supply vector). The programmer is deterministic, validated against the structured model, and isolates the LLM-based planner from low-level naming and indexing details. Because the patch planner usually emits canonical operations directly, the programmer typically performs only minor index coercion; its role is most visible when a natural-language label or representation does not match the model's internal naming.

**Agent 2: Re-optimization Strategy Selector.** Given the normalized action sets, the strategy selector chooses a re-optimization strategy from a toolbox of computational techniques. Let $\mathcal{A}_t$ denote the set of available re-optimization strategies at time $t$, which may include *direct warm starts* from a saved base solution, *heuristic warm starts* produced by a problem-specific construction, *tuned solver configurations* obtained from offline tuning, *variable-fixing rules*, *valid inequalities*, and *prior-solution* reuse. The selector maps the action sets, the affected components, previous-solution information, and the computational budget to a strategy

$$a_t = \texttt{SelectReoptStrategy}(\widetilde{\Pi}_t, \mathcal{R}_t, \mathcal{Z}_{t-1}, x_{t-1}, \mathcal{A}_t).$$

Here $x_{t-1}$ denotes the saved solution associated with $\mathcal{Z}_{t-1}$, if available. This selector is separated from the patch planner because a correct model edit and an effective re-optimization strategy address different tasks: the former determines whether the requested model change has been expressed correctly, while the latter determines how the updated model should be solved within the time budget.

**Agent 3: Validator and Optimization Engine.** For each normalized action set $\Pi \in \widetilde{\Pi}_t$ and selected strategy $a_t$, this module (1) applies the patch sequence $\mathcal{Z}^\Pi = (\mathcal{M}^\Pi, \mathcal{P}^\Pi, p^\Pi) = \Pi(\mathcal{Z}_{t-1})$, (2) regenerates the solver model from $\mathcal{M}^\Pi$ and $p^\Pi$, (3) configures the optimization run using $a_t$, (4) solves

$$x_t^\Pi \in \arg\min_{x \in \mathcal{X}(\mathcal{Z}^\Pi)} F(x; \mathcal{Z}^\Pi),$$

and (5) returns feasibility, objective value, and diagnostics on constraint violations. These diagnostics also provide structured feedback to the end user regarding whether the generated modifications satisfy the intended operational request and preserve implementability under the updated conditions. Across the candidate action sets, the framework retains the action set with the lowest

objective among the prompt-satisfying solutions.

**Validation-triggered retry.** A failure at any stage, including unparseable planner output, patch-application errors, infeasible solves, or violations of prompt-specific constraints, is captured as a typed failure record and fed back to the patch planner as repair context. The planner is then re-invoked, conditioned on the failure record, and the resulting candidate action sets are processed by the programmer and Agents 2–3 again. This repair loop is bounded by a fixed retry budget (a small integer in the implementation). Retries are reported as reliability diagnostics, separating cases that succeed on the first attempt from those that require repair. This retry mechanism acts as a reliability layer that helps maintain robustness and correctness under imperfect natural-language interpretation.

### 5.5 Closed-Loop Re-Optimization

At each time step $t$, the **ReOpt-LLM** framework executes the following procedure, with the inner repair loop bounded by a retry budget $B$:

1. Initialize repair context $\rho \leftarrow \emptyset$ and attempt counter $n \leftarrow 0$.
2. While $n < B$ and no candidate action set has succeeded:
   (a) $(E_t, \mathcal{R}_t, \Pi_t) \leftarrow \texttt{Plan}(\Delta_t, \mathcal{Z}_{t-1}, \rho)$ via the integrated patch planner.
   (b) $\widetilde{\Pi}_t \leftarrow \texttt{Normalize}(\Pi_t, \mathcal{Z}_{t-1})$ via the programmer.
   (c) Set $a_t \leftarrow \texttt{SelectReoptStrategy}(\widetilde{\Pi}_t, \mathcal{R}_t, \mathcal{Z}_{t-1}, x_{t-1}, \mathcal{A}_t)$.
   (d) $(\Pi_t^*, x_t^*, \rho) \leftarrow \texttt{ValidateAndSolve}(\widetilde{\Pi}_t, a_t, \mathcal{Z}_{t-1})$, returning the best prompt-satisfying action set and solution, or a failure record in $\rho$ if no candidate succeeds.
   (e) $n \leftarrow n + 1$.
3. If no candidate action set succeeds, report $\rho$, set $\mathcal{Z}_t \leftarrow \mathcal{Z}_{t-1}$, and keep $x_t \leftarrow x_{t-1}$.
4. Otherwise, set $\mathcal{Z}_t \leftarrow \Pi_t^*(\mathcal{Z}_{t-1})$ and store $x_t \leftarrow x_t^*$.

The closed-loop procedure continually aligns the structured model representation with evolving operational conditions and user-specified rules, and the retry mechanism makes the framework robust to occasional planner errors.

The framework requires only (i) a structured state $\mathcal{Z}_t = (\mathcal{M}_t, \mathcal{P}_t, p_t)$, (ii) natural-language change descriptions $\Delta_t$, and (iii) access to an optimization solver. It therefore applies broadly to transportation and flow problems, production scheduling with vehicle assignment, exam timetabling, and other large-scale optimization problems that must be continuously adapted and re-optimized as operational conditions and decision requirements evolve.

## 6 Toy Example

This section illustrates the framework on a small transportation instance. The example provides an interpretable execution trace showing how natural-language operational requests are translated into structured model edits, normalized, validated, and re-optimized through the full workflow. The resulting solutions are then compared against the baseline plan to confirm that the closed loop preserves feasibility, satisfies the intended operational request, and provides an interpretable trace of how the model was modified.

Consider a classical transportation problem where a set of plants must supply goods to a set of customers. Let $I$ denote the set of plants, indexed by $i$, and $J$ the set of customers, indexed by $j$. Each plant $i \in I$ has a limited supply $s_i$, and each customer $j \in J$ requires a demand quantity $d_j$. Shipping one unit of product from plant $i$ to customer $j$ incurs a transportation cost $\kappa_{ij}$. The decision variables $x_{ij}$ represent the quantity shipped from $i$ to $j$. The mathematical formulation is

$$\min \quad \sum_{i \in I} \sum_{j \in J} \kappa_{ij}\, x_{ij} \tag{6.1}$$

$$\text{s.t.} \quad \sum_{j \in J} x_{ij} \le s_i, \qquad \forall\, i \in I, \tag{6.2}$$

$$\sum_{i \in I} x_{ij} \ge d_j, \qquad \forall\, j \in J, \tag{6.3}$$

$$x_{ij} \ge 0, \qquad \forall\, i \in I,\, j \in J. \tag{6.4}$$

Objective (6.1) minimizes the total transportation cost incurred when shipping goods from plants to customers. Constraints (6.2) enforce the supply capacity at each plant, ensuring that total outgoing shipments do not exceed available supply. Constraints (6.3) guarantee that each cus-

tomer's demand requirement is satisfied through inbound shipments. Finally, Constraints (6.4) restrict all shipment quantities to be nonnegative. In the structured model representation of Section 5.1, this problem has one variable family $\mathcal{V} = \{\texttt{flows}\}$ with index set $I \times J$, two constraint families $\mathcal{C} = \{\texttt{supply_constraints}, \texttt{demand_constraints}\}$, and one objective component $\mathcal{O} = \{\texttt{transport_cost}\}$. The named parameter set is $\mathcal{P} = \{\texttt{supply}, \texttt{demand}, \texttt{costs}\}$, with values collected in $p$ and descriptive metadata attached to each entry for retrieval.

The toy instance contains two plants and three customers. The supply capacities are $s_1 = 20$ and $s_2 = 45$; the customer demands are $d_1 = 12$, $d_2 = 15$, and $d_3 = 18$. Transportation costs follow the matrix

$$\kappa = \begin{pmatrix} 4 & 6 & 8 \\ 5 & 4 & 3 \end{pmatrix}.$$

The baseline transportation model achieves a minimum total cost of 162.0. In this solution, Plant 1 supplies 12 units to Customer 1, while Plant 2 supplies 15 units to Customer 2 and 18 units to Customer 3. The allocation satisfies every demand and respects both supply capacities. Each prompt below is fed to the workflow starting from this baseline.

### 6.1 Prompt 1: Plant Maintenance

**User prompt.** *"Plant 1 is going into urgent maintenance for the next two days, so it cannot ship anything."*

**Patch Planner (Agent 1).** In a single LLM call, the planner emits the structured event with $S_t = \{\texttt{plants} : [\texttt{P1}]\}$ and edit summary *"set supply of plant P1 to zero to represent urgent maintenance downtime"*, the relevant components $\mathcal{R}_t = \{\texttt{supply}, \texttt{supply_constraints}, \texttt{flows}\}$, and one candidate action set containing the single patch `UPDATE_PARAMETER` on `supply` with key `P1` and value 0.0. The two-day maintenance window is captured in the edit summary but not in the patch itself, since the toy model is single-period.

**Programmer.** The proposed patch is already canonical: the entity label `P1` matches the registered identifier and the parameter key is correctly typed. The programmer leaves the patch unchanged. Because the patch is explicit and localized, the end user can directly verify that the proposed edit correctly reflects the operational maintenance request before re-optimization.

**Strategy Selector (Agent 2).** The selector picks a warm strategy, with the recorded rationale that the edit sets one plant's supply to zero, expected reuse is high, and warm start is supported and available.

**Validator and Optimization Engine (Agent 3).** The patch is applied, the solver model is regenerated, and the solve completes feasibly. The total cost increases from 162 to 174, with all shipments rerouted through Plant 2.

The closed loop produces a feasible re-optimized plan that satisfies the user's maintenance request by eliminating all outbound shipments from Plant 1 while preserving demand feasibility. The cost increase reflects the loss of Plant 1's lower-cost capacity.

### 6.2 Prompt 2: Transportation Shortage

**User prompt.** *"There is an unexpected shortage of trucks for deliveries from Plant 2 to Customer 2 this week. The maximum that can be shipped on this route is 5 units."*

**Patch Planner (Agent 1).** The planner emits the structured event with $S_t = \{\texttt{plants} : [\texttt{P2}], \texttt{customers} : [\texttt{C2}]\}$ and edit summary *"limit shipment flow from Plant 2 to Customer 2 to a maximum of 5 units due to truck shortage"*, relevant components $\mathcal{R}_t = \{\texttt{flows}\}$, and one candidate action set containing the single patch `UPDATE_BOUND` on `flows` with index `[P2, C2]`, bound type `upper`, and value 5.0.

**Programmer.** The programmer promotes the index list `[P2, C2]` to the tuple `(P2, C2)` expected by the `flows` family. No structural rewrite is needed; the patch op remains `UPDATE_BOUND`.

**Strategy Selector (Agent 2).** A warm strategy is selected, with the rationale that the route cap is a local change, expected reuse is high, and no structural modification is introduced.

**Validator and Optimization Engine (Agent 3).** The bound update is applied, and re-optimization redirects shipments from the capped route. The solve is feasible with total cost 184, and the flow on $(\texttt{P2}, \texttt{C2})$ saturates at 5; Plant 1 absorbs more of Customer 2's demand while Plant 2 covers the remaining Customer 1 demand needed to respect Plant 1's supply capacity.

The framework absorbs the route-level capacity restriction while satisfying the user's request that shipments on route $(\texttt{P2}, \texttt{C2})$ never exceed the imposed limit of 5 units. It also does not modify any

constraint family, illustrating that local capacity edits can be expressed as bound updates rather than as constraint-RHS edits.

### 6.3 Prompt 3: Customer Order

**User prompt.** *"Customer 3 has placed an urgent order of 10 additional units on top of their normal demand."*

**Patch Planner (Agent 1).** The planner emits the structured event with $S_t = \{\texttt{customers} : [\texttt{C3}]\}$ and edit summary *"increase demand for customer C3 by 10 units due to urgent order"*, relevant components $\mathcal{R}_t = \{\texttt{demand}, \texttt{demand_constraints}\}$, and one candidate action set containing the single patch `UPDATE_PARAMETER` on `demand` with key `C3` and a delta of +10. By emitting the additive change as a `delta` field rather than an absolute value, the planner pushes the arithmetic onto the patch executor and avoids hand-computing the new total.

**Programmer.** The proposed patch is canonical and uses the additive form supported by the patch language. The programmer leaves the patch unchanged.

**Strategy Selector (Agent 2).** A warm strategy is selected, with the rationale that the demand increase is local, expected reuse is high, and no structural edit is introduced.

**Validator and Optimization Engine (Agent 3).** The patch is applied, increasing the demand at `C3` from 18 to 28, and the solve completes feasibly with total cost 192. The increase reflects the additional shipments required to satisfy the urgent order.

The example illustrates how additive natural-language requests are absorbed at the parameter level via the `delta` update form while ensuring that the increased demand for Customer 3 is fully satisfied in the re-optimized solution. Letting the patch language carry the additive semantics keeps the patch planner free to operate at the component level and avoids placing arithmetic responsibilities on the LLM.

## 7 Evaluation Protocol

This section operationalizes the abstract LLM evaluation dimensions of Section 4 into the concrete experimental protocol used in both case studies. The framework variants, LLM models, design grid,

retry mechanism, success criteria, validation procedures, and failure-mode taxonomy are common to the OCP and Cornell evaluations. Time limit, toolbox composition, and quality metric are deliberately case-specific because they reflect the managerial setting, and are described in the respective case-study experimental design subsections.

### 7.1 Framework Variants

The evaluation compares three variants of the framework, holding the rest of the pipeline fixed.

***ReOpt-LLM-Patch.*** The proposed implementation. The LLM expresses the user request through the structured patch language of Section 5.3, the LLM selector chooses re-optimization techniques from the case-specific toolbox, and the validator executes the resulting model update.

***Direct-Code Agent.*** A baseline in which the LLM edits implementation code directly rather than using the structured patch language. This variant shares the validator and retry budget, but its outputs are arbitrary code edits rather than explicit, auditable, and reviewable patch operations.

***ReOpt-LLM-Patch without Selector.*** An ablation that retains the structured patch representation but removes the toolbox-selection step. The model update is applied and re-solved without any toolbox elements.

The toolbox composition itself is case-specific and is described in each case-study section.

### 7.2 LLM Models, Design Grid, and Retry

Each case study evaluates the three variants on three OpenAI models (OpenAI, 2026): gpt-4.1-mini, gpt-4.1, and gpt-5. Each case provides five instances and six prompt classes, so the design contains $5 \times 6 \times 3 \times 3 = 270$ LLM-assisted prompt-instance cases (instances $\times$ prompts $\times$ models $\times$ variants). Each model-by-variant row of an aggregate comparison table therefore aggregates 30 prompt-instance cases.

To support detailed instance-by-prompt reporting, each case study designates one configuration as its *default* and reports a $5 \times 6$ instance-by-prompt table for that configuration. The defaults are *ReOpt-LLM-Patch* with gpt-4.1-mini for OCP and *ReOpt-LLM-Patch* with gpt-5 for Cornell. All other configurations contribute only to the aggregate baseline and ablation comparisons.

All LLM-assisted runs share a validation-triggered retry mechanism with a uniform budget of one repair attempt. When the first attempt produces a patch that fails schema or semantic validation, the framework returns the validator's error feedback to the LLM and requests a single corrected attempt. This retry mechanism acts as a controlled reliability layer, allowing the framework to recover from imperfect natural-language interpretations while preserving validation consistency across all configurations.

## 7.3 Success Criteria and Failure-Mode Taxonomy

A prompt-instance case is classified along the four nested success criteria in Table 7.1 that evaluate whether the framework correctly interprets, validates, and operationally satisfies the user request. Each criterion implies the success of the criteria above it, so reported rates in aggregate comparison tables are non-increasing across the four criteria.

| **Criterion** | **Definition** |
|---|---|
| Update correctness | The generated model edits correctly and transparently matches the intended operational update expressed in the prompt. The ground-truth update is reference-derived in each case study. |
| Prompt satisfaction | Update correctness, and additionally, a feasible recovered plan that satisfies the intended operational request while respecting all prompt-specific constraints. |
| First-attempt success | Update correctness and prompt satisfaction achieved on the first attempt. |
| Final success | Update correctness and prompt satisfaction achieved after the retry budget is applied. |

Table 7.1: Nested success criteria used in both case studies.

Failures are classified using the fixed taxonomy in Table 7.2 to support consistent operational diagnosis, interpretability, and reliability analysis across both case studies. The modes are not mutually exclusive: a single case may exhibit several modes simultaneously.

## 7.4 Shared Computational Environment

Both case studies run on nodes with 2 Intel Xeon Gold 6226 processors at 2.7 GHz and 24 GB of memory per allocated node, using Python 3.11 with Gurobi/gurobipy 13.0.1. Reported solve time is wall-clock time measured inside the Gurobi optimization call. The number of CPU cores and threads, the per-case time limit, the toolbox composition, the quality metric, and the default LLM

| Mode | Definition |
|---|---|
| *Wrong comp.* | The LLM targeted the wrong model component. For direct-code runs this is folded into *Bad update*, since the direct-code agent does not expose a structured component-targeting step. |
| *Invalid patch* | The patch fails schema or application validation. |
| *Bad update* | The applied update does not match the intended edit. |
| *No incumbent* | The solver returned no feasible solution within the time limit (equivalently, no validated incumbent was accepted). |
| *Prompt viol.* | A feasible incumbent was returned but its edit violates at least one prompt-specific operational constraint. |
| *Missing output* | No usable validated framework output is returned. |

Table 7.2: Failure-mode taxonomy used uniformly in both case studies.

configuration are case-specific and are stated in the experimental design subsections of Sections 8 and 9.

# 8 OCP Group Case Study

This section presents the OCP case study, which focuses on large-scale downstream production and shipment scheduling in a complex real-world industrial supply chain. The section first summarizes the underlying optimization problem at the Jorf site, then describes the experimental design used to evaluate the proposed framework, reports the main computational results, and concludes with the resulting managerial insights.

## 8.1 Problem Description

The OCP case study focuses on downstream supply-chain operations at the Jorf site in Morocco, one of the world's largest phosphate-processing and export facilities. The optimization model is a large-scale mixed-integer linear program that integrates production, storage, and vessel-loading decisions over a monthly planning horizon with daily resolution. The model captures the physical flow of materials across a highly interconnected network of conveyors, pipelines, storage units, and quays, while accounting for a demand-driven order-fulfillment process and a large product portfolio. Key decisions include selecting shipments and loading schedules, developing daily production plans across multiple processing units, and determining the implied inventory and changeover trajectories. Given the scale and combinatorial complexity of the problem, the framework supports coordinated downstream scheduling and provides actionable insights for operational planning at a site respon-

sible for a large share of the group's production. The original industrial study provides additional background on the model and operating context (Er Raqabi et al., 2025).

## 8.2 Experimental Design

**Instances.** The OCP evaluation considers five realistic monthly instances: $OCP_1$, $OCP_2$, $OCP_3$, $OCP_4$, and $OCP_5$. Table 8.1 summarizes the season, planning horizon, number of vessels, total demand, and model size for each instance. In the computational results below, the fulfillment denominator counts the shipments that remain active after preprocessing for a given prompt-instance case, and may therefore differ slightly from the raw vessel count reported in the instance table.

| ID | Season | Horizon | Vessels | Demand | Variables | Binaries | Constraints |
|---|---|---|---|---|---|---|---|
| **$OCP_1$** | Winter | 30 | 38 | 846,702 | 125,880 | 16,292 | 109,977 |
| **$OCP_2$** | Winter | 30 | 38 | 856,686 | 126,314 | 16,695 | 110,922 |
| **$OCP_3$** | Autumn | 32 | 62 | 2,031,400 | 947,598 | 17,966 | 695,693 |
| **$OCP_4$** | Summer | 24 | 40 | 1,043,330 | 298,693 | 33,284 | 235,344 |
| **$OCP_5$** | Autumn | 32 | 61 | 957,338 | 948,009 | 18,264 | 780,267 |
| **Avg** | | **29.6** | **47.8** | **1,147,091** | **489,299** | **20,500** | **386,441** |

Table 8.1: OCP instances over different seasons.

**Prompts.** The OCP prompt catalog contains six representative classes of online operational updates: demand adjustment, shipment arrival delay, weather disruption, production-capacity reduction, transportation-capacity degradation, and a composed prompt that combines demand and delay. These six prompt classes span demand-side, transportation-side, production-side, quay-side, and multi-update perturbations, and therefore provide a compact but representative testbed for online re-optimization. Table 8.2 gives one example for each prompt class.

**Re-optimization toolbox.** Following the evaluation protocol of Section 7, the detailed instance-by-instance results below use the OCP default configuration: *ReOpt-LLM-Patch* with gpt-4.1-mini. The OCP toolbox is designed for online industrial re-optimization, where resilience and speed are both important (Er Raqabi et al., 2024). Its main components are a history of solutions, a set of customized heuristics, and a set of instance-specific tuned Gurobi configurations. In the selector-enabled variant, the LLM chooses a legal combination of these toolbox elements before the modified model is re-solved.

| Prompt | Example |
|---|---|
| $P_1$ | *Shipment 13297 requested an increase of 1500 tonnes to its initial demand of 4000 tonnes.* |
| $P_2$ | *The vessel corresponding to Shipment 13297 will have a delay and has a new expected arrival date of January 20th.* |
| $P_3$ | *The weather is bad on January 12th, and no loading operations are maintained on that day.* |
| $P_4$ | *Production Unit 0280 suffers from an unexpected breakdown and requires maintenance on the 17th, operating only 12 hours instead of 24.* |
| $P_5$ | *The conveyors linking the storage point with Quai 1 will operate at 80% of their nominal capacity.* |
| $P_6$ | *Apply both updates together: Shipment 13297 requests an increase of 1500 tonnes, and the vessel corresponding to Shipment 13297 has a new expected arrival date of January 20th.* |

Table 8.2: Examples of user prompts for OCP re-optimization.

**Computational environment.** In addition to the shared settings of Section 7.4, the OCP solves used 8 CPU cores with Gurobi configured to use 8 threads. Each OCP re-optimization run was subject to a 300-second time limit to reflect the online decision-support setting.

**Validation and metrics.** Because the OCP setting is online, the primary performance criteria are correct model adaptation, feasible recovery, high shipment fulfillment, and fast runtime (Er Raqabi et al., 2023, 2025). Patch-based runs are validated against reference OCP edits using the success criteria and failure-mode taxonomy of Section 7.3. The main reported metrics are fulfillment, solve time, and solver gap, where the gap is the solver-reported relative MIP gap for the internal OCP objective, not a gap in shipment fulfillment. Direct-code runs are assessed by feasible completion and the same metrics, but they do not expose the same structured semantic trace as the patch-based runs.

## 8.3 Computation Results

**Headline performance.** Table 8.3 reports the case-by-case results for the default *ReOpt-LLM-Patch* configuration on the five OCP instances and six prompt classes. Across the full $5\times6$ evaluation grid, the framework returns a feasible incumbent for all 30 cases. The generated patch matches the reference edit on all 30 cases; prompt-satisfying recovery is achieved on 29 of 30. The only prompt-level failure is $\text{OCP}_5$–$P_1$: the demand-update patch is applied correctly, but re-optimization under the 300-second limit returns only 7 of 58 active shipments (versus reference recovery with substan-

tially higher fulfillment). That incumbent is reported for diagnostic completeness. Among prompt-satisfying cases, the most difficult recovery is $OCP_5$–$P_4$, which fulfills 49 of 58 active shipments with a 25.96% solver gap. These results show that the framework is operationally robust across the evaluation grid, while severe perturbations can still reduce the number of active shipments that can be fulfilled within the time cap.

| Instance | Prompt | Update correct | Prompt satisfied | Fulfilled | Time (s) | Gap (%) |
|---|---|---|---|---|---|---|
| **$OCP_1$** | P1 | ✓ | ✓ | 37/38 | 37 | 0.00% |
| **$OCP_1$** | P2 | ✓ | ✓ | 38/38 | 4 | 0.00% |
| **$OCP_1$** | P3 | ✓ | ✓ | 37/38 | 300 | 3.67% |
| **$OCP_1$** | P4 | ✓ | ✓ | 38/38 | 49 | 0.00% |
| **$OCP_1$** | P5 | ✓ | ✓ | 38/38 | 53 | 0.00% |
| **$OCP_1$** | P6 | ✓ | ✓ | 37/38 | 36 | 0.00% |
| **$OCP_2$** | P1 | ✓ | ✓ | 37/38 | 107 | 0.00% |
| **$OCP_2$** | P2 | ✓ | ✓ | 38/38 | 3 | 0.00% |
| **$OCP_2$** | P3 | ✓ | ✓ | 38/38 | 110 | 0.00% |
| **$OCP_2$** | P4 | ✓ | ✓ | 38/38 | 107 | 0.00% |
| **$OCP_2$** | P5 | ✓ | ✓ | 38/38 | 160 | 0.00% |
| **$OCP_2$** | P6 | ✓ | ✓ | 37/38 | 107 | 0.00% |
| **$OCP_3$** | P1 | ✓ | ✓ | 59/60 | 98 | 0.00% |
| **$OCP_3$** | P2 | ✓ | ✓ | 60/60 | 131 | 0.00% |
| **$OCP_3$** | P3 | ✓ | ✓ | 60/60 | 152 | 0.00% |
| **$OCP_3$** | P4 | ✓ | ✓ | 60/60 | 12 | 0.00% |
| **$OCP_3$** | P5 | ✓ | ✓ | 60/60 | 12 | 0.00% |
| **$OCP_3$** | P6 | ✓ | ✓ | 59/60 | 99 | 0.00% |
| **$OCP_4$** | P1 | ✓ | ✓ | 37/38 | 85 | 0.00% |
| **$OCP_4$** | P2 | ✓ | ✓ | 38/38 | 112 | 0.00% |
| **$OCP_4$** | P3 | ✓ | ✓ | 38/38 | 100 | 0.00% |
| **$OCP_4$** | P4 | ✓ | ✓ | 38/38 | 4 | 0.42% |
| **$OCP_4$** | P5 | ✓ | ✓ | 38/38 | 4 | 0.42% |
| **$OCP_4$** | P6 | ✓ | ✓ | 37/38 | 89 | 0.21% |
| **$OCP_5$** | P1 | ✓ | × | 7/58 | 300 | 3.65% |
| **$OCP_5$** | P2 | ✓ | ✓ | 58/58 | 14 | 0.00% |
| **$OCP_5$** | P3 | ✓ | ✓ | 55/58 | 300 | 3.96% |
| **$OCP_5$** | P4 | ✓ | ✓ | 49/58 | 300 | 25.96% |
| **$OCP_5$** | P5 | ✓ | ✓ | 58/58 | 13 | 0.00% |
| **$OCP_5$** | P6 | ✓ | ✓ | 58/58 | 18 | 0.00% |

Table 8.3: Case-by-case OCP results for the default *ReOpt-LLM-Patch* configuration. *Fulfilled* reports fulfilled shipments over the active post-preprocessing denominator for that prompt-instance case.

**Baseline comparison.** Table 8.4 compares the structured patch formulation with direct code editing across the three LLMs. In this setting, *update correctness* requires that the generated model edit match the ground-truth OCP update, while *prompt satisfaction* additionally requires a feasible recovered plan. Since no configuration attains an additional semantic success through retry,

first-attempt and final success coincide in this comparison. The structured patch representation is consistently more reliable than direct code editing: *ReOpt-LLM-Patch* attains final success rates of 96.7%, 93.3%, and 93.3% for gpt-4.1-mini, gpt-4.1, and gpt-5, respectively, whereas the direct-code baseline does not produce a semantically correct update in any of the evaluated cases. In these cases, the direct-code agent often produces executable or partially executable modifications, but the resulting model does not match the reference OCP edit semantics, so the solver is frequently applied to the wrong modified problem. These results indicate that, for OCP, the main advantage of the patch interface is semantic reliability rather than feasibility alone.

| Model | Method | Update correctness | Prompt satisfaction | First-attempt success | Final success |
|---|---|---|---|---|---|
| gpt-4.1-mini | *Direct-Code Agent* | 0.0 | 0.0 | 0.0 | 0.0 |
| gpt-4.1-mini | *ReOpt-LLM-Patch* | 100.0 | 96.7 | 96.7 | 96.7 |
| gpt-4.1 | *Direct-Code Agent* | 0.0 | 0.0 | 0.0 | 0.0 |
| gpt-4.1 | *ReOpt-LLM-Patch* | 100.0 | 96.7 | 96.7 | 96.7 |
| gpt-5 | *Direct-Code Agent* | 0.0 | 0.0 | 0.0 | 0.0 |
| gpt-5 | *ReOpt-LLM-Patch* | 100.0 | 96.7 | 96.7 | 96.7 |

Table 8.4: Baseline comparison between the *Direct-Code Agent* and *ReOpt-LLM-Patch* in the OCP evaluation. Each row aggregates 30 prompt-instance cases. Entries are percentages and follow the success criteria of Section 7.3.

**Failure analysis.** Table 8.5 summarizes the failure patterns underlying the method comparison. For patch-based runs, failures are sparse and concentrated in a small number of cases, most notably a recurring demand-update miss on $\text{OCP}_5$–$P_1$, which appears as an invalid patch application. By contrast, the direct-code baseline more often returns solver-feasible plans whose edits do not match the intended ground-truth update. As a result, the direct-code rows are dominated by *bad update* and prompt-level semantic violations rather than by pure optimization failure. This distinction explains why the direct-code baseline may still recover incumbents with nontrivial fulfillment while failing to achieve semantic success.

| Model | Variant | Wrong comp. | Invalid patch | Bad update | No incumbent | Prompt viol. | Missing output |
|---|---|---|---|---|---|---|---|
| gpt-4.1-mini | *Direct-Code Agent* | 0 | 0 | 30 | 3 | 27 | 3 |
| gpt-4.1-mini | *ReOpt-LLM-Patch* | 1 | 1 | 1 | 1 | 0 | 1 |
| gpt-4.1 | *Direct-Code Agent* | 0 | 0 | 30 | 5 | 25 | 5 |
| gpt-4.1 | *ReOpt-LLM-Patch* | 2 | 1 | 2 | 2 | 0 | 2 |
| gpt-5 | *Direct-Code Agent* | 0 | 0 | 30 | 1 | 29 | 1 |
| gpt-5 | *ReOpt-LLM-Patch* | 2 | 1 | 2 | 2 | 0 | 2 |

Table 8.5: Failure modes for the *Direct-Code Agent* and *ReOpt-LLM-Patch* in the OCP evaluation. *Note.* Each entry counts prompt-instance cases (out of 30) that exhibit the corresponding failure mode. Columns follow the taxonomy of Table 7.2.

**Selector ablation.** Table 8.6 shows the effect of the LLM-guided toolbox selector for the default OCP configuration. The table reports feasible completion rather than semantic success. Under that operational criterion, both variants return an incumbent on all 30 cases, but selector guidance materially improves operational performance: mean fulfillment increases from 86.57% to 95.65%, mean runtime falls from 194.88 seconds to 97.20 seconds, and mean solver gap decreases from 2.24% to 1.28%. Mean fulfillment is reported as an unweighted average of the 30 case-level fulfillment percentages. Among the 29 semantically valid recoveries, mean fulfillment increases further to 98.54%, indicating that the low overall mean is driven primarily by the single rejected $\text{OCP}_5$–$P_1$ diagnostic run. In the online OCP setting, the main value of toolbox selection is therefore not binary solve success alone, but faster recovery and higher-quality shipment plans under tight time limits.

| Variant | Feasible comp. | No incumbent | Invalid / build fail. | Mean fulfill. (%) | Mean time (s) | Mean gap |
|---|---|---|---|---|---|---|
| ReOpt-LLM-Patch (selector), gpt-4.1-mini | 100.0% | 0 | 0 | 95.65% | 97.20 | 1.28% |
| ReOpt-LLM-Patch without selector, gpt-4.1-mini | 100.0% | 0 | 0 | 86.57% | 194.88 | 2.24% |

Table 8.6: Ablation of the LLM-guided re-optimization toolbox selector for the default OCP configuration. Lower runtime and lower gap are better.

**Operational-quality diagnostics.** The semantic failure table above explains why the structured patch interface is necessary, but it does not replace the online quality story. In the default configuration, $\text{OCP}_5$–$P_1$ is the lone semantic failure, while $\text{OCP}_5$–$P_4$ is the hardest semantically valid operational case, with fulfillment dropping to 49/58 and the solver gap rising to 25.96%. These cases motivate the use of fulfillment, runtime, and gap as the main online evaluation criteria in addition to semantic correctness.

## 8.4 Managerial Insights

The OCP case highlights the managerial value of LLM-assisted re-optimization in large-scale industrial environments subject to continuous operational disruptions. In practice, planners must rapidly react to demand changes, vessel delays, bad weather, unit breakdowns, or transportation-capacity degradation while preserving as much of the accepted operational plan as possible. The managerial challenge is therefore not only solving a large-scale optimization model, but continuously adapting the deployed decision-support system as business conditions evolve. The relevant benchmark is thus whether the system can quickly interpret operational disruptions, correctly update the optimization

model, recover feasible shipment plans, and maintain high fulfillment under strict operational time limits.

The results show that the structured patch-based framework provides this operational reliability. In the default OCP configuration, *ReOpt-LLM-Patch* returns a feasible incumbent on all thirty prompt-instance pairs, achieves semantically valid prompt-satisfying recovery on 29 of the 30 cases, and maintains 95.65% mean fulfillment across all raw incumbents. Among the 29 semantically valid recoveries, mean fulfillment increases to 98.54%. Importantly, planners can express disruptions directly in natural language while obtaining validated and auditable model updates without requiring continuous intervention from the optimization engineering team. This is managerially significant because many deployed optimization systems gradually lose effectiveness as operational rules evolve faster than models can be manually maintained.

The structured patch representation is particularly important from a governance and implementation perspective because it creates a transparent and traceable link between the user request, the affected model components, the applied optimization edits, and the final solution. This improves interpretability, facilitates validation, and increases organizational trust in AI-assisted optimization systems.

The toolbox selector provides a complementary operational benefit. Even when multiple variants recover feasible plans, selector-guided re-optimization produces higher fulfillment, shorter runtimes, and lower solver gaps under the same online time budget. This distinction matters operationally because a technically feasible plan with poor fulfillment or slow recovery may still be unacceptable in practice. More broadly, the case illustrates how LLM-assisted re-optimization can reduce decision latency in complex industrial environments by transforming natural-language disruption reports into validated optimization edits and rapid re-solves.

# 9 Cornell University Case Study

This section presents the Cornell University exam scheduling case study, illustrating the application of the **ReOpt-LLM** framework to a large-scale real-world academic scheduling problem. It first describes the exam scheduling optimization model and its key constraints, then outlines the experi-

mental design, presents the computational results, and concludes with managerial insights relevant to university administrators.

## 9.1 Problem Description

The second case study is drawn from the final exam scheduling problem at Cornell University, where an integer programming framework was developed to address the complex and highly constrained nature of university-wide exam timetabling. The model accommodates a wide range of institutional requirements, including the front-loading of large courses, the exclusion of specific time slots, and policies governing exam conflicts. By generating and comparing multiple model variants and incorporating heuristic solution approaches, the framework supports informed decision-making by the university registrar, allowing explicit trade-offs between schedule quality and student conflicts. The approach demonstrated clear improvements over traditional lecture time-based scheduling methods, yielding substantial administrative time savings and increased satisfaction among students and faculty (Ye et al., 2026).

The evaluation focuses on the block-sequencing stage of the Cornell framework. Exam blocks are treated as already constructed by the block-assignment stage, and the re-optimization task assigns these fixed blocks to exam time slots under updated policy and schedule-quality requirements. The block-sequencing MIP enforces assignment and continuity constraints so that each exam block is placed once and each slot receives a valid block sequence. It also exposes policy constraint families that are natural targets for re-optimization. Front-loading constraints require large exam blocks to be scheduled before an instance-specific cutoff slot. Reserved-slot requests are handled through virtual blocks. Fixing a virtual block to a reserved slot keeps that slot unavailable to real exam blocks while preserving the one-block-per-slot sequencing structure. Day-level capacity constraints can be added by aggregating block enrollments over the slots in a given day. The objective penalizes weighted counts of undesirable student exam patterns, including same-day triples, triples within 24 hours, evening-to-morning back-to-back exams, other back-to-back exams, and three exams in four consecutive slots.

This case differs from the OCP setting in two important ways. First, re-optimization is offline, i.e., the registrar can evaluate alternative schedules before publication. Consequently, the revised

| Instance | Semester | # Exams | # Slots | # Blocks | # Binary | # Constraints |
|---|---|---|---|---|---|---|
| **$\mathbf{EXAM_1}$** | Spring 2024 | 544 | 24 | 20 | 677,376 | 401,011 |
| **$\mathbf{EXAM_2}$** | Fall 2023 | 601 | 24 | 18 | 677,376 | 401,010 |
| **$\mathbf{EXAM_3}$** | Spring 2023 | 553 | 25 | 17 | 796,875 | 468,866 |
| **$\mathbf{EXAM_4}$** | Fall 2022 | 588 | 24 | 19 | 677,376 | 401,010 |
| **$\mathbf{EXAM_5}$** | Spring 2022 | 539 | 24 | 16 | 677,376 | 401,006 |
| **Avg** | | **565** | **24** | **18** | **701,276** | **414,581** |

Table 9.1: Cornell exam-scheduling instances.

schedule need not remain close to a previously implemented plan. Second, given that the re-optimization is offline, the primary managerial objective is schedule quality rather than speed. The evaluation, therefore, assesses **ReOpt-LLM** by whether it correctly interprets scheduling-policy prompts, applies the intended model update, satisfies the resulting constraints, and produces schedules whose objective values are close to a saved hindsight reference incumbent obtained under the same time limit.

## 9.2 Experimental Design

**Instances.** The evaluation uses five calibrated synthetic Cornell exam-scheduling instances generated with the Cornell final-exam scheduling MIP generator. The generator builds block-assignment and block-sequencing MIPs using anonymized co-enrollment, triplet co-enrollment, and exam-size inputs, and provides a synthetic data generator for larger calibrated instances. The formulations evaluated here are block-sequencing MIPs, with the block assignment taken as fixed. Table 9.1 reports the paper-facing instance identifiers together with the semester label, number of exams, available time slots, exam blocks, binary variables, and constraints used for each instance. All variables in these generated formulations are binary.

**Prompts.** The exam prompt catalog contains six representative prompt classes that capture policy changes and late operational updates faced during exam scheduling. These prompts cover reserved time slots, late co-enrollment changes, large-exam deadlines, stress-related objective-weight adjustments, day-level proctoring capacity, and a composed update that applies multiple policy changes in sequence. Table 9.2 gives one example for each prompt class. The large-exam deadline in prompt $P_3$ is instance-specific. Large exams must be completed before slot 20 for $\text{EXAM}_1$, slot 19 for

| Prompt | Example |
|---|---|
| $P_1$ | *Reserve the evening slot immediately before the final evening slot so the staff can begin arranging the auditorium for graduation events.* |
| $P_2$ | *Increase the pairwise co-enrollment count between Block 4 and Block 9 by 120 students due to late add/drop changes.* |
| $P_3$ | *Ensure all large exams with over 300 students are completed before the instance-specific cutoff time slot to allow teaching assistants sufficient grading time.* |
| $P_4$ | *The Student Assembly raised concerns about extreme stress; increase the penalty for having "three exams in 24 hours" to be 20 times that of a regular back-to-back.* |
| $P_5$ | *Due to an unexpected shortage of available proctors, limit the total number of students taking exams on Day 2 to a maximum of 4,000.* |
| $P_6$ | *A combination of prompts, e.g., $P_4$, then $P_2$, then $P_1$.* |

Table 9.2: Examples of user prompts for Cornell exam-scheduling re-optimization.

$\text{EXAM}_2$, slot 17 for $\text{EXAM}_3$, slot 19 for $\text{EXAM}_4$, and slot 15 for $\text{EXAM}_5$. Prompt $P_6$ exercises the composition of three updates that target disjoint parts of the model: $P_4$ modifies objective weights, $P_2$ modifies co-enrollment parameters, and $P_1$ modifies slot availability. The composed model is invariant to the order in which these updates are applied, so the listed sequence is illustrative.

**Re-optimization toolbox.** Following the evaluation protocol of Section 7, the detailed instance-by-prompt and schedule-quality results below use the Cornell default configuration: *ReOpt-LLM-Patch* with gpt-5. The exam-scheduling toolbox consists of three reusable elements that can be applied individually or in combination. *Direct warm start* uses the saved base block-sequencing solution as a Gurobi MIP start for the modified model. *Heuristic warm start* builds a greedy schedule after the prompt-specific edit by first fixing the virtual-block assignments required for reserved slots and then placing constrained and low-enrollment blocks to respect front-loading and day-load-cap restrictions. The resulting schedule is converted to Gurobi variable starts. *Tuned configuration* applies instance-specific Gurobi parameter files obtained from the tuning workflow. In *ReOpt-LLM-Patch*, the LLM selector chooses a feasible executable combination of these elements for each prompt-instance case, such as *direct+heuristic+tuned*. The scratch label is not a toolbox element. It is used only in *ReOpt-LLM-Patch without Selector* to disable the selector and force a solve without any toolbox elements.

**Computational environment.** In addition to the shared settings of Section 7.4, the Cornell solves used 6 CPU cores with Gurobi configured to use 6 threads. Each Cornell re-optimization run was subject to a 3600-second time limit to reflect the offline pre-publication setting.

**Validation and metrics.** Because the Cornell setting is offline, the primary performance criterion is schedule quality at publication time. Patch-based runs are validated against deterministic prompt-specific reference-script edits using the success criteria and failure-mode taxonomy of Section 7.3. The main reported metrics are the objective difference $\Delta\text{obj} = \text{obj}_{\text{LLM}} - \text{obj}_{\text{Ref}}$ and the corresponding reference-relative percentage gap against a saved hindsight reference incumbent solved with the *direct+heuristic+tuned* configuration under the same 3600-second time limit. A negative reference-relative gap on a valid, prompt-satisfying case means the framework found a better incumbent than the hindsight reference, while negative gaps from invalid configurations are not interpreted as quality improvements. Schedule quality is additionally reported using the metrics of Ye et al. (2026): triples, back-to-back exams, two exams in 24 hours, and three exams in four slots. Direct conflicts, also defined in Ye et al. (2026), are zero in every evaluated case and are omitted from the reported tables.

## 9.3 Computation Results

**Headline performance.** Table 9.3 reports the instance-by-prompt results for the default *ReOpt-LLM-Patch* configuration. The framework satisfies all 30 prompts and produces no prompt-specific constraint violations. Nineteen of the 30 prompt-instance cases match the saved hindsight reference incumbent exactly. The largest reference-relative deviations occur for prompt classes $P_2$ and $P_4$, which modify the co-enrollment structure and objective weights, respectively. These cases are expected to be more sensitive to the interaction between the re-optimization technique and the solver's incumbent search under the 3600-second cap.

Figure 3 visualizes the reference-relative objective gaps for these prompt-instance cases. Most cases are at or near zero, while the few larger deviations are concentrated in the more structurally consequential prompts. The figure reinforces the main conclusion from Table 9.3. *ReOpt-LLM-Patch* consistently produces valid, prompt-satisfying schedules and usually remains close to the saved hindsight reference incumbent under the same time budget.

**Schedule-quality details.** Table 9.4 aggregates the schedule-quality metrics by prompt and compares them with the saved 3600-second hindsight reference incumbents. The aggregate deltas are small relative to the scale of the instances.

| Instance | Prompt | Update correct | Prompt satisfied | ReOpt obj. | Ref. obj. | Δobj |
|---|---|---|---|---|---|---|
| $EXAM_1$ | P1 | ✓ | ✓ | 6,901 | 6,901 | 0 |
| $EXAM_1$ | P2 | ✓ | ✓ | 6,662 | 5,338 | 1,324 |
| $EXAM_1$ | P3 | ✓ | ✓ | 8,615 | 8,615 | 0 |
| $EXAM_1$ | P4 | ✓ | ✓ | 7,756 | 5,938 | 1,818 |
| $EXAM_1$ | P5 | ✓ | ✓ | 8,871 | 8,871 | 0 |
| $EXAM_1$ | P6 | ✓ | ✓ | 9,325 | 9,325 | 0 |
| $EXAM_2$ | P1 | ✓ | ✓ | 7,336 | 7,336 | 0 |
| $EXAM_2$ | P2 | ✓ | ✓ | 6,331 | 6,331 | 0 |
| $EXAM_2$ | P3 | ✓ | ✓ | 9,719 | 9,576 | 143 |
| $EXAM_2$ | P4 | ✓ | ✓ | 6,834 | 6,847 | -13 |
| $EXAM_2$ | P5 | ✓ | ✓ | 6,410 | 6,410 | 0 |
| $EXAM_2$ | P6 | ✓ | ✓ | 8,306 | 8,204 | 102 |
| $EXAM_3$ | P1 | ✓ | ✓ | 4,915 | 4,915 | 0 |
| $EXAM_3$ | P2 | ✓ | ✓ | 4,003 | 4,003 | 0 |
| $EXAM_3$ | P3 | ✓ | ✓ | 9,468 | 9,468 | 0 |
| $EXAM_3$ | P4 | ✓ | ✓ | 4,233 | 4,233 | 0 |
| $EXAM_3$ | P5 | ✓ | ✓ | 4,150 | 4,150 | 0 |
| $EXAM_3$ | P6 | ✓ | ✓ | 5,409 | 5,409 | 0 |
| $EXAM_4$ | P1 | ✓ | ✓ | 6,488 | 6,488 | 0 |
| $EXAM_4$ | P2 | ✓ | ✓ | 5,648 | 5,648 | 0 |
| $EXAM_4$ | P3 | ✓ | ✓ | 8,452 | 8,306 | 146 |
| $EXAM_4$ | P4 | ✓ | ✓ | 6,215 | 6,194 | 21 |
| $EXAM_4$ | P5 | ✓ | ✓ | 5,849 | 5,779 | 70 |
| $EXAM_4$ | P6 | ✓ | ✓ | 7,376 | 7,376 | 0 |
| $EXAM_5$ | P1 | ✓ | ✓ | 2,645 | 2,645 | 0 |
| $EXAM_5$ | P2 | ✓ | ✓ | 2,560 | 2,537 | 23 |
| $EXAM_5$ | P3 | ✓ | ✓ | 9,237 | 9,237 | 0 |
| $EXAM_5$ | P4 | ✓ | ✓ | 2,534 | 2,528 | 6 |
| $EXAM_5$ | P5 | ✓ | ✓ | 2,570 | 2,570 | 0 |
| $EXAM_5$ | P6 | ✓ | ✓ | 2,674 | 2,674 | 0 |

Table 9.3: Cornell exam-scheduling results for the default *ReOpt-LLM-Patch* configuration. The objective difference $\Delta\text{obj} = \text{ReOpt obj.} - \text{Ref. obj.}$ is reported so that exact matches with the hindsight reference incumbent ($\Delta\text{obj} = 0$) can be read directly. Reference-relative percentage gaps are visualized in Figure 3, and per-prompt schedule-quality metrics in Table 9.4.

**Baseline comparison.** Table 9.5 compares *ReOpt-LLM-Patch* with the *Direct-Code Agent*. For each model, the patch-based implementation has a higher final success rate than direct code editing: 63.3% versus 3.3% for gpt-4.1-mini, 83.3% versus 26.7% for gpt-4.1, and 100% versus 33.3% for gpt-5. The model comparison also shows that stronger LLMs improve reliability, but model strength alone does not remove the need for the structured framework. The *Direct-Code Agent* remains below 50% final success even with gpt-5, indicating that direct implementation-code editing introduces grounding and validation challenges that are not resolved by simply using a stronger model.

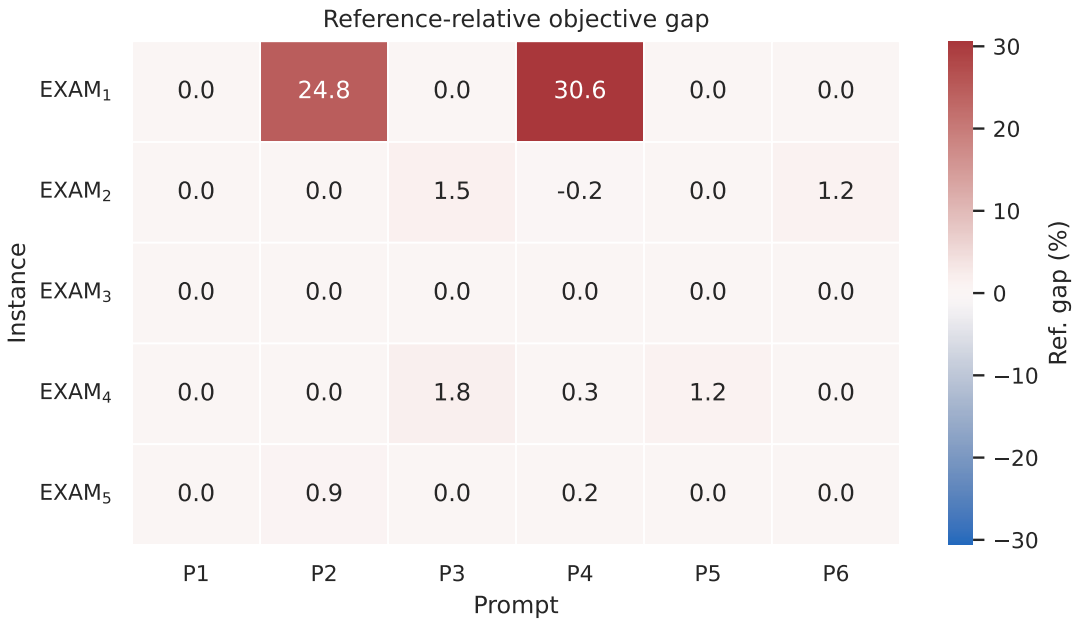



Figure 3: Reference-relative objective gap for the default *ReOpt-LLM-Patch* configuration.

| Metric | Series | Overall | P1 | P2 | P3 | P4 | P5 | P6 |
|---|---|---|---|---|---|---|---|---|
| Triples | ReOpt | 126.5 | 109.8 | 88.8 | 237.8 | 89.4 | 118.2 | 115.0 |
| Triples | Ref. | 120.6 | 109.8 | 77.2 | 233.6 | 73.6 | 115.8 | 113.8 |
| Triples | Δ | 5.9 | 0 | 11.6 | 4.2 | 15.8 | 2.4 | 1.2 |
| Back-to-back | ReOpt | 3,338.9 | 3,267.0 | 3,028.8 | 4,229.2 | 3,044.4 | 3,119.0 | 3,345.0 |
| Back-to-back | Ref. | 3,319.7 | 3,267.0 | 2,974.4 | 4,226.4 | 2,987.0 | 3,120.0 | 3,343.6 |
| Back-to-back | Δ | 19.2 | 0 | 54.4 | 2.8 | 57.4 | -1.0 | 1.4 |
| Two in 24 hours | ReOpt | 10,399.6 | 10,245.0 | 9,892.4 | 12,167.2 | 9,742.6 | 9,916.8 | 10,433.6 |
| Two in 24 hours | Ref. | 10,383.7 | 10,245.0 | 9,843.8 | 12,166.0 | 9,620.4 | 9,989.0 | 10,438.2 |
| Two in 24 hours | Δ | 15.9 | 0 | 48.6 | 1.2 | 122.2 | -72.2 | -4.6 |
| Three in four slots | ReOpt | 603.5 | 549.0 | 484.2 | 1,001.6 | 466.8 | 517.4 | 601.8 |
| Three in four slots | Ref. | 593.0 | 549.0 | 446.8 | 994.4 | 446.2 | 520.8 | 600.8 |
| Three in four slots | Δ | 10.5 | 0 | 37.4 | 7.2 | 20.6 | -3.4 | 1.0 |

Table 9.4: Schedule-quality comparison for the default *ReOpt-LLM-Patch* configuration. Each cell aggregates the metric across the five Cornell instances. *ReOpt* is the LLM-assisted result, *Ref.* is the saved hindsight reference incumbent, and Δ is their difference.

**Retry diagnostics.** The first-attempt and final success columns of Table 9.5 characterize the retry mechanism's contribution. The default *ReOpt-LLM-Patch* configuration with gpt-5 succeeds on the first attempt throughout the evaluation and does not require retries. For weaker configurations, the retry mechanism provides a small reliability layer: gpt-4.1-mini *ReOpt-LLM-Patch* improves from 60.0% to 63.3%, gpt-5 *Direct-Code Agent* from 26.7% to 33.3%, and gpt-4.1-mini *Direct-Code Agent* from 0.0% to 3.3% after retries are applied.

| Model | Method | Update correctness | Prompt satisfaction | First-attempt success | Final success |
|---|---|---|---|---|---|
| gpt-4.1-mini | *Direct-Code Agent* | 13.3 | 3.3 | 0.0 | 3.3 |
| gpt-4.1-mini | *ReOpt-LLM-Patch* | 63.3 | 63.3 | 60.0 | 63.3 |
| gpt-4.1 | *Direct-Code Agent* | 30.0 | 26.7 | 26.7 | 26.7 |
| gpt-4.1 | *ReOpt-LLM-Patch* | 83.3 | 83.3 | 83.3 | 83.3 |
| gpt-5 | *Direct-Code Agent* | 36.7 | 33.3 | 26.7 | 33.3 |
| gpt-5 | *ReOpt-LLM-Patch* | 100.0 | 100.0 | 100.0 | 100.0 |

Table 9.5: Baseline comparison between the *Direct-Code Agent* and *ReOpt-LLM-Patch* in the Cornell evaluation. *Note.* Each row aggregates 30 prompt-instance cases and entries are the percentage that satisfy the corresponding criterion. The four nested success criteria are defined in Section 7.3.

**Failure analysis.** Table 9.6 summarizes the failure modes behind the direct-code and structured-patch comparison. Direct code editing most often fails because code changes cannot be grounded cleanly back to the intended scheduling operation, or because the resulting update satisfies only part of the prompt. For example, the *Direct-Code Agent* has 25, 21, and 19 prompt-specific violations for gpt-4.1-mini, gpt-4.1, and gpt-5, respectively. The structured patch interface reduces these failures by forcing the LLM output into auditable model-edit operations before optimization.

| Model | Variant | Wrong comp. | Invalid patch | Bad update | No incumbent | Prompt viol. | Missing output |
|---|---|---|---|---|---|---|---|
| gpt-4.1-mini | *Direct-Code Agent* | 2 | 1 | 26 | 2 | 25 | 2 |
| gpt-4.1-mini | *ReOpt-LLM-Patch* | 2 | 0 | 11 | 0 | 11 | 0 |
| gpt-4.1 | *Direct-Code Agent* | 0 | 0 | 21 | 0 | 21 | 0 |
| gpt-4.1 | *ReOpt-LLM-Patch* | 0 | 0 | 5 | 0 | 5 | 0 |
| gpt-5 | *Direct-Code Agent* | 5 | 3 | 19 | 5 | 19 | 5 |
| gpt-5 | *ReOpt-LLM-Patch* | 0 | 0 | 0 | 0 | 0 | 0 |

Table 9.6: Failure modes for the *Direct-Code Agent* and *ReOpt-LLM-Patch* in the Cornell evaluation. *Note.* Each entry counts prompt-instance cases (out of 30) that exhibit the corresponding failure mode. Columns follow the taxonomy of Table 7.2. General-violation and retry-exhausted counts are zero throughout and are omitted.

**Selector ablation.** Table 9.7 compares *ReOpt-LLM-Patch* with *ReOpt-LLM-Patch without Selector* for the default exam-scheduling configuration. The ablation preserves the structured patch plan but forces a solve without any toolbox elements or their combinations. Holding the model-update representation fixed, LLM-guided toolbox selection raises final success from 83.3% to 100%, eliminates five no-incumbent cases, and reduces the median objective difference from 1,441 to 0 (median reference-relative gap from 25.7% to 0.0%) under the same 3600-second time limit. The mean objective difference falls from 3,577,316.3 to 121.3, but is dominated by four no-selector cases in which the time limit expired with the solver still in a high-triple-penalty incumbent region. This pattern supports a two-part interpretation of the framework. Patch editing improves semantic control over

the model update. In contrast, toolbox selection improves the reliability and incumbent quality of the subsequent re-optimization.

| Variant | Final success | No incumbent | Mean Δobj | Median Δobj | Mean ref. gap | Median ref. gap |
|---|---|---|---|---|---|---|
| *ReOpt-LLM-Patch* | 100.0% | 0 | 121.3 | 0.0 | 2.1% | 0.0% |
| *ReOpt-LLM-Patch without Selector* | 83.3% | 5 | 3,577,316.3 | 1,441.0 | 88,459.9% | 25.7% |

Table 9.7: Effect of the LLM-guided toolbox selector on the default exam-scheduling configuration. Each row aggregates 30 prompt-instance cases for gpt-5. Objective and gap summaries exclude no-incumbent cases. Lower values are better in every column.

Table 9.8 extends the comparison across all three models on prompt-instance cases where both variants produced prompt-satisfying incumbents: toolbox selection produces a lower objective on 13 of 16 cases for gpt-4.1-mini, 16 of 20 for gpt-4.1, and 22 of 25 for gpt-5, with median objective improvements above 1,400 in every model.

| Model | Common valid cases | Selector lower obj. | Median obj. improvement | Mean obj. improvement |
|---|---|---|---|---|
| gpt-4.1-mini | 16 | 13/16 | 1,430.5 | 5,588,645.4 |
| gpt-4.1 | 20 | 16/20 | 1,430.5 | 4,471,130.5 |
| gpt-5 | 25 | 22/25 | 1,441.0 | 3,577,173.0 |

Table 9.8: Paired objective improvement from LLM-guided toolbox selection in the Cornell evaluation. Each row restricts attention to prompt-instance cases for which both *ReOpt-LLM-Patch* and *ReOpt-LLM-Patch without Selector* produced prompt-satisfying incumbents. Objective improvement is no-selector minus selector. Positive values favor the selector variant.

## 9.4 Managerial Insights

The Cornell case highlights the value of LLM-assisted re-optimization in settings where the organization must repeatedly adjust an unpublished plan before committing to it. Before the final exam schedule is released, the registrar can revise the model in response to late policy changes, stakeholder concerns, or capacity information without requiring a full return to the original OR modeling team. This shifts the operational workflow from a sequential, expert-dependent process to an interactive and decentralized decision process in which end users can directly adapt the model. The relevant standard is therefore not proximity to the previous schedule, but whether the modified model is auditable, feasible, and capable of producing a high-quality schedule before publication. In this offline setting, decision quality dominates speed, and the ability to explore and validate multiple alternatives before commitment becomes the primary managerial objective.

A key implication is the reduction in decision latency and the expansion of decision-making capa-

bilities. In traditional settings, incorporating new constraints or policy updates may require multiple iterations between stakeholders and OR experts, often taking days. The proposed framework enables the registrar to evaluate multiple what-if scenarios within the same decision cycle, effectively transforming re-optimization into an interactive planning tool rather than a batch process. This capability is particularly valuable in academic scheduling, where late updates are frequent and where evaluating alternative policies, such as stress reduction or capacity adjustments, is essential before finalizing the schedule.

The structured patch interface is central to this managerial value. It creates a traceable chain from the natural-language prompt to the affected model component, the model update, the validation checks, and the final schedule. This audit trail is especially important for academic scheduling, where students, faculty, or administrators may contest policy changes. Beyond transparency, this traceability serves as a governance mechanism, reducing the risk of unintended or inconsistent model modifications and facilitating communication across stakeholders. The results show that the default *ReOpt-LLM-Patch* configuration satisfies all tested prompt classes, produces no prompt-specific violations, and remains close to the saved hindsight reference incumbent on average.

More broadly, the case illustrates how LLM-assisted re-optimization reduces dependence on OR experts while preserving solution reliability, thereby enabling a separation between policy specification and model implementation. The retry mechanism provides an additional reliability layer for weaker configurations, although the default configuration succeeds on the first attempt throughout the evaluation. These insights extend beyond academic scheduling to other pre-commitment planning settings, where organizations must iteratively refine decisions before execution, highlighting the potential of LLM-based systems to enhance adaptability, accountability, and decision quality in complex operational environments.

## 10 Conclusions

This paper proposes an LLM-assisted collaborative re-optimization framework, **ReOpt-LLM**, that integrates end users and optimization to enable adaptive decision support. The framework formalizes how user prompts, reflecting new rules or operational changes, translate into structured and auditable updates of the core optimization model, enabling the system to evolve dynamically with

changing environments. By embedding this process within a language interface, the framework lowers the barrier between domain users and large-scale mathematical optimization models while preserving solver-based validation and mathematical rigor. Methodologically, the paper highlights that large-scale re-optimization is fundamentally a structured model-reasoning problem rather than a mere re-solving task. The proposed framework combines interpretable patch-based model edits with solver-aware re-optimization strategies driven by a toolbox of OR techniques, including historical solutions, valid inequalities, solver configurations, heuristics, and ML-guided components. The computational results further show that structured model-edit interfaces and toolbox-aware orchestration significantly improve reliability and solution quality compared with direct code-editing approaches. Managerially, the framework enables organizations to sustain and adapt deployed optimization systems without constant expert intervention, thereby improving the long-term sustainability of decision-support tools in dynamic operational settings. Future work may extend this foundation to stochastic, robust, or multiobjective settings and evaluate its performance across additional large-scale applications. Ultimately, the framework advances the vision of sustainable, human-centered optimization systems that remain adaptive, interpretable, and operationally effective under continuous change.

## Acknowledgments

This research was partly supported by the NSF AI Institute for Advances in Optimization (Award 2112533).

## Appendix A Framework Artifacts

This appendix documents the agent prompts, the patch JSON schema, the validator decision logic, and the direct-code baseline used in both case studies.

### A.1 Agent System Prompts

The pipeline uses three LLM agents in sequence: a Patch Planner, a Strategy Selector, and an optional Code-Edit Planner used only by the direct-code baseline of Appendix A.3.

### A.1.1 Patch Planner

The Patch Planner converts a user prompt $\Delta_t$ into a candidate set of structured patches. Its system instruction is reproduced below.

**Patch Planner – system instruction**

You are a reoptimization planner. Use the deterministic model representation to interpret the requested change and propose candidate model edits. Return JSON only. Use `candidate_action_sets` as the canonical output format.

Each `candidate_action_set` is one executable plan; put all coordinated edits for a single plan in the same action set. Each item must be a JSON object with `actions=[...]`; the patch list key inside each item must be `actions`. Each patch object must use the canonical keys `op`, `target`, `scope`, `update`, and optional `notes`. Return multiple candidate action sets only when they are genuinely different alternative plans, and do not duplicate the same edits in both grouped and flat forms.

Patch payloads must be executable as written: use concrete ids and numeric literals for indices, row labels, and values whenever the model representation provides enough information. Do not emit pseudo-code, formulas, set names, or symbolic placeholders inside patch indices or values. For keyed parameter edits, place the concrete sub-index in `update.key`. For numeric requests phrased as "increase by", "decrease by", or other additive changes, use `update.delta` instead of overwriting with `update.value`. For `materialized_linear` constraint families, use matching concrete row ids in `lhs_spec.rows` and `rhs_spec`, and concrete executable variable indices in every term. If the representation explicitly exposes a compact problem-specific semantic `lhs_spec.kind`, that semantic payload may be used instead of materializing every row term. If a valid executable patch cannot be expressed, return empty candidate lists rather than a symbolic or guessed placeholder patch.

Required JSON keys: `edit_summary` (short free-form summary of the requested edit); `affected_sets` (object mapping entity or set labels to identifiers mentioned or strongly implied by the delta, or `{}` if none); `relevant_components` (list of model component names); `candidate_action_sets` (list of executable candidate plans, each `{actions: [...]}`); `planning_hints` (optional planner hints such as `edit_scope='local|structural'` or `expected_reuse='high|low'`).

At runtime, the framework appends to this system message a per-case block of problem-specific guidance (Appendices A.1.3 and A.1.4), the list of allowed patch operators for the case, generic operator guidance, and the patch schemas of Table 5.1. On validation failure, a structured repair-context block is added to the user message of the next planning call, prefaced by the prelude

reproduced below.

**Patch Planner – repair-context prelude (retry only)**

This is a fresh repair attempt for the same user request from a fresh planning pass. Preserve the user's intent, not the previous implementation details. Use the runtime feedback below only to avoid the previous failure mode. The items below are runtime feedback from earlier attempts in this same run.

The repair-context block itself consists of bullet entries derived from the fields `failure_stage`, `failure_kind`, `failure_message`, `repair_instruction`, and an `attempt_history` list of recent failures, under the one-attempt retry budget of Section 7.2.

### A.1.2 Strategy Selector

The Strategy Selector reads the validated patch set and the case-specific toolbox catalog, then emits a JSON object with the required keys `solve_strategy`, `toolbox_plan`, `rationale`, and the optional key `confidence` $\in [0, 1]$.

**Strategy Selector – system instruction**

You choose the fastest safe reoptimization solve strategy. Return JSON only.

Pick exactly one solve strategy from the allowed list. Do not invent toolbox items or unsupported strategies. Toolbox plans are executable in this runtime and must match the chosen solve strategy.

Prefer `warm+tuned` over `warm` alone when both warm reuse and tuned solving are available and the edit looks reuse-friendly. Prefer warm reuse for local edits when a reusable solution exists but tuned solving is unavailable or unnecessary. Prefer tuned or scratch for structural edits when warm reuse looks fragile.

Required JSON keys: `solve_strategy`, `toolbox_plan`, `rationale`. Optional JSON key: `confidence` as a number in $[0, 1]$.

### A.1.3 OCP Case-Specific Framing

The OCP setup injects a problem-context block into every Patch-Planner call. The block describes the OCP supply-chain model (its objective, variable, and constraint naming conventions, and date encoding) so the planner can interpret prompts and emit regex-driven patches against the pre-compiled LP. Unlike the Cornell setup, OCP does not inject a separate patch-form guidance block. The three OCP-applicable operations (the pattern-based family in Table 5.1) and their schemas are

surfaced through the generic operator-guidance mechanism.

**OCP domain framing – problem context**

You are working with a large-scale mixed-integer linear program (MILP) for the OCP Group downstream supply-chain at the Jorf site in Morocco. The model integrates production, storage, and vessel-loading decisions over a monthly planning horizon with daily resolution. The model is provided as a pre-compiled LP file; you do NOT have access to the Python source code that generated it. All modifications must be expressed as structured patches that operate on variable and constraint names via regex patterns.

**Objective.** The model maximizes total shipment fulfillment (`TotalFulfillment`). Secondary objectives (changeover minimization, lateness penalties, etc.) are present but weighted at 0 in the base formulation.

**Variable naming conventions.** All variable names end with a suffix of the form `_(<numeric_id>)#<integer>`. These suffixes are instance-specific and must be matched via regex, never hard-coded. Use the prefix (everything before the first parenthesis) for pattern matching.

**Constraint naming conventions.** Constraint names also end with instance-specific suffixes. The match is by prefix.

### A.1.4 Cornell Case-Specific Framing

The Cornell setup injects two complementary domain blocks into every Patch-Planner call. The first describes the exam-scheduling model, its calendar, objective, core variables, and grounding data, so the planner can interpret prompts in the Registrar's vocabulary. The second pins each prompt class to a canonical exam patch form, preventing the planner from inventing alternative constraint-family names for behaviors that the exam model already supports.

**Cornell domain framing – problem context**

You are helping the University Registrar evaluate changes to the final-exam schedule. This is the block-sequencing stage of a Group-then-Sequence workflow: exams have already been grouped into blocks, and this model places those blocks into exam slots. The model is about exam timing only; room assignment is handled separately. Delta requests usually describe policy, comfort, or operational changes to the exam calendar.

**Calendar and basic assumptions.** The exam period is a fixed ordered sequence of slots. The standard interpretation is three slots per day – morning, afternoon, and evening, typically `9am`, `2pm`, and `7pm`.

Requests like "Day 2", "morning", or "the evening slot immediately before the final evening slot" should be grounded through `slots_per_day`, `slot_times`, and slot ids, not through guessed LP names. Some slots may be intentionally excluded from use; in final patch proposals, relative calendar phrases should be resolved to explicit slot ids when the instance data makes that possible.

**Objective.** The sequencing objective penalizes stressful student exam patterns: `alpha` (triples within one day), `beta` (triples within 24 hours), `gamma1` (evening-to-morning back-to-backs), `gamma2` (other back-to-backs), and `delta` (three exams in four consecutive slots). These events are not double-counted: exam pairs already part of a triple are not also counted as back-to-backs.

**Core model view.** The sequencing formulation is cyclic over the slot set. `x[i,j,k,s]=1` means block $i$ is placed at slot $s$, block $j$ at slot $s+1$, and block $k$ at slot $s+2$; assignment-like policy rules are expressed by summing `x[i,*,*,s]` terms over the relevant slots. The variables `y[...]` and `z[...]` are linkage variables used to score triple and four-slot patterns. Assignment and continuity constraints enforce a valid cyclic schedule in which each block is placed once and each slot receives one block.

**Grounding data.** The rendered model representation already exposes `virtual_blocks`, `large_blocks`, `early_slots`, `reserved_slots`, `block_enrollment`, `pair_counts`, `triplet_counts`, `slots_per_day`, and `slot_times`. Use those rendered values directly instead of inventing symbolic placeholders, and use the exact numeric slot cutoff stated in the prompt when front-loading requests are instance-specific.

**Canonical interpretations.** Slot-reservation requests use `reserved_virtual_slot`; front-loading requests are interpreted through `large_blocks` and `early_slots`; day-load requests use `block_enrollment` and the first-index occupancy view of `x[i,j,k,s]`; co-enrollment changes usually affect `pair_counts` or `triplet_counts`, where pairwise block relationships are unordered, so both ordered pair keys are updated unless the request explicitly distinguishes direction; weight or comfort tradeoff requests usually affect `alpha`, `beta`, `gamma1`, `gamma2`, or `delta`. When the instance data resolves a slot or day directly, explicit slot ids are emitted rather than symbolic formulas.

**Combined requests.** The requested order is preserved when one request bundles several changes; combined Registrar edits may mix weight changes, co-enrollment edits, front-loading, slot reservations, and day-load restrictions.

**Cornell domain framing – patch-form guidance**

Use the canonical exam patch forms. Do not invent new constraint-family names or semantic `lhs` kinds when an existing exam family fits.

For slot exclusion requests like the penultimate evening slot, emit `ADD_CONSTRAINT_FAMILY` with `constraint.name='reserved_virtual_slot'`, `lhs_spec.kind='reserved_virtual_slot'`, the explicit grounded slot id, and the smallest available virtual block id.

For large-exam frontloading requests, emit `UPDATE_PARAMETER(name='early_slots', value=[...])` with explicit slot ids; do not invent a new frontload family name.

For pairwise co-enrollment changes between two blocks, emit keyed `UPDATE_PARAMETER` patches for both ordered pair keys `[a,b]` and `[b,a]` on the pair-count override surface unless the request explicitly distinguishes direction.

For day-level load caps, emit `ADD_CONSTRAINT_FAMILY` with `constraint.name='slot_load_cap'`, `lhs_spec.kind='slot_load_cap'`, explicit slot ids, and a stable row id such as `'day_2'`.

## A.2 Validator Decision Logic

The validator is a best-improvement candidate solver. For each candidate patch in the planner's output:

1. Apply the patch to a copy of the base model. Catch schema or application errors and record them as a failure entry. On success, proceed.
2. Invoke the Gurobi solver under the case-specific time limit, providing a warm start when the Strategy Selector has specified one.
3. Distinguish between (i) a clean infeasible or no-incumbent solve and (ii) unexpected errors, which surface the original exception.
4. Compare returned incumbents across surviving candidates and retain the one with the lowest objective value.

If all candidates fail, the validator returns the structured feedback consumed by the retry mechanism. The validator does not perform a separate semantic check on the LLM's intent. Semantic correctness is judged by whether the executed patch produces a feasible incumbent that satisfies the prompt-specific constraints encoded by the Update-correctness criterion of Section 7.3.

## A.3 Direct-Code Agent Specification

The direct-code baseline delegates code editing to the open-source Aider CLI (Gauthier, 2024). Its system instruction is reproduced below.

**Direct-Code Agent – system instruction**

You are editing a packaged re-optimization problem through Aider. Modify only the editable files provided to you; use any read-only files only as context. Keep the code executable and minimal. Prefer changing the canonical solver implementation that is rebuilt and re-solved. Honor the natural-language request and any generic problem guidance provided below.

When the surfaced editing surface is an LP-wrapper, as in the OCP case, the system instruction is extended with two LP-wrapper rules.

**Direct-Code Agent – LP-wrapper extension**

For LP-wrapper editing, `build_codeedit_model(...)` must only mutate and return a Gurobi model. Do not solve the model or inspect solution values inside `build_codeedit_model(...)`.

A case-specific code-edit guidance block is appended for both case studies, keeping the agent inside the existing solver surface rather than rewriting model structure.

**Direct-Code Agent – OCP code-edit guidance**

For LP-backed codeedit runs, edit only `lp_codeedit_wrapper.py`. Keep the wrapper thin and generic, operate on actual LP variable and constraint names via regex, and do not invent new `runtime_data` keys or instance-specific helper APIs.

**Direct-Code Agent – Cornell code-edit guidance**

Use the existing exam solver mechanisms rather than inventing new interfaces or rewriting the model structure. For slot exclusion requests, resolve the concrete slot from `slot_times` and `slots_per_day`, update `reserved_slots`, and preserve the virtual-block reservation mechanism that materializes the slot exclusion. For pairwise co-enrollment changes between two blocks, keep pair-count updates symmetric across both ordered keys unless the request explicitly distinguishes direction. For large-exam frontloading requests, reuse `early_slots` and the existing large-blocks logic instead of adding a new frontload formulation. For day-level load caps, ground the limit with `block_enrollment` over the affected day slots and preserve the existing slot-load-cap semantics. For stress-penalty changes, update the existing objective weight parameters rather than rewriting the objective structure.

The agent's action space is the set of unified diffs that Aider can apply to the surfaced editable files, and its output is the captured unified diff together with the run status. The same one-attempt retry budget is applied: a failed compilation or solve raises the validator's feedback back to Aider,

which produces a corrected diff. Aider is invoked with the same OpenAI model under evaluation, so the only protocol-level difference between *ReOpt-LLM-Patch* and *Direct-Code Agent* is the action representation – a structured patch versus an arbitrary code edit.

# Appendix B Methodology Details

## B.1 Gurobi Tuning Workflow

Both case studies use Gurobi's built-in tuning tool to produce instance-specific parameter files. Tuning is performed once per instance on the *base* (unperturbed) model, before any prompt is applied. The same tuned parameter file is then reused across all prompts, framework variants, and LLM models for that instance, including the LLM-assisted runs and the hindsight reference run. This ensures that no variant or prompt sees a parameter file tuned for its specific perturbation, so the tuned configuration acts as a fixed instance-level baseline rather than as a per-case advantage.

The time budgets differ between the two case studies. For OCP, the Gurobi `TuneTimeLimit` is set to 28,800 seconds (8 hours) and each tuning candidate is given a 1,800-second (30-minute) solve budget before `model.tune()` is invoked. For Cornell, the `TuneTimeLimit` is set to 86,400 seconds (24 hours), with no per-candidate solve cap set explicitly.

## B.2 OCP Re-optimization Heuristics

The OCP re-optimization heuristics are inspired by the rollback-and-repair strategy of Er Raqabi et al. (2024), and are designed to generate high-quality warm starts from a previously optimized solution under localized disruptions. The general principle is to preserve as much of the incumbent solution structure as possible while selectively relaxing decisions that are directly affected by the prompt-induced changes. This ensures fast recovery and strong initial incumbents for the subsequent MIP solve.

At a high level, the heuristic proceeds in two steps. First, the solver retrieves the set of decision variables that are active in the base solution, which represents a complete feasible plan before disruption. Second, it applies prompt-dependent filtering rules that identify which parts of the solution are invalidated by the new operational context (e.g., demand changes, delays, capacity

reductions, weather disruptions, or production constraints). Variables unaffected by the disruption are fixed to their incumbent values to preserve solution structure, while those directly impacted are released to allow re-optimization. The resulting partially fixed solution is then passed as a warm start to Gurobi, enabling rapid recovery of feasibility and improved incumbent quality under the new constraints.

### B.3 Exam-Scheduling Heuristic Warm Start

The exam-scheduling heuristic warm start constructs a feasible block-to-slot assignment that respects prompt-induced policy changes before passing it to Gurobi as a MIP start. The construction proceeds in four stages, summarized in Algorithm 1: reserved-slot pinning, front-loading of large exams, day-load capping, and a default fallback to the cached base assignment.

The notation is as follows. $\mathcal{B}$ is the block set with enrollment function $e : \mathcal{B} \to \mathbb{N}$. $\mathcal{S}$ is the slot set, partitioned by exam day as $\mathcal{S} = \bigsqcup_{d \in \mathcal{D}} \mathcal{S}_d$, where $\mathcal{D}$ is the set of exam days. The prompt parameter tuple $\theta = (\theta_{\text{rsv}}, \tau, c, \theta_{\text{cap}})$ specifies a partial reserved-slot map $\theta_{\text{rsv}}$ for virtual blocks, a large-exam enrollment threshold $\tau$, a front-loading cutoff slot $c$, and a partial day-enrollment cap $\theta_{\text{cap}}$. The cached base assignment from the prior solve is $X^0 : \mathcal{B} \to \mathcal{S}$. The algorithm builds the output $X$ incrementally, using a free-slot pool $F \subseteq \mathcal{S}$ that tracks the slots not yet assigned to any block. A block is *unassigned* if it is not in $\text{dom}(X)$. Ties in any sort are broken by block id in ascending order.

## References


AhmadiTeshnizi, A., Gao, W., and Udell, M. (2024). OptiMUS: Scalable optimization modeling using MIP solvers and large language models. In *International Conference on Machine Learning (ICML)*.

Ai, R., Barbalho, H. D. O., Li, S., Robsky, A., Simchi-Levi, D., and Menache, I. (2025). Solve smart, not often: Policy learning for costly milp re-solving. *arXiv preprint arXiv:2509.23470*.

Archetti, C., Guastaroba, G., and Speranza, M. G. (2013). Reoptimizing the rural postman problem. *Computers & Operations Research*, 40(5):1306–1313.

Austin, D., Korikov, A., Toroghi, A., and Sanner, S. (2024). Bayesian optimization with llm-based acquisition functions for natural language preference elicitation. In *Proceedings of the 18th ACM Conference on Recommender Systems*, pages 74–83.

Bang, S. and Song, H. (2025). LLM-based user profile management for recommender system. *arXiv preprint arXiv:2502.14541*.

**Algorithm 1:** Exam-scheduling heuristic warm start.

**Input:** Block set $\mathcal{B}$, slot set $\mathcal{S} = \bigsqcup_{d \in \mathcal{D}} \mathcal{S}_d$, parameters $\theta = (\theta_{\text{rsv}}, \tau, c, \theta_{\text{cap}})$, base assignment $X^0 : \mathcal{B} \to \mathcal{S}$.

**Output:** Warm-start assignment $X : \mathcal{B} \to \mathcal{S}$ supplied to Gurobi as a MIP start.

$X \leftarrow \emptyset$; $F \leftarrow \mathcal{S}$ `// F is the free-slot pool`

`// Stage 1: pin reserved slots`

**foreach** $v \in \mathrm{dom}(\theta_{\text{rsv}})$ **do**
  $s \leftarrow \theta_{\text{rsv}}(v)$; $X(v) \leftarrow s$; $F \leftarrow F \setminus \{s\}$
**end**

`// Stage 2: front-load large exams`

**foreach** *unassigned* $b \in \mathcal{B}$ *with* $e(b) \geq \tau$*, in non-increasing* $e(b)$ *order* **do**
  $P \leftarrow F \cap \{s \in \mathcal{S} : s < c\}$ `// free pre-cutoff slots`
  **if** $P \neq \emptyset$ **then**
    $s \leftarrow \min P$; $X(b) \leftarrow s$; $F \leftarrow F \setminus \{s\}$
  **end**
**end**

`// Stage 3: enforce day-load caps`

**foreach** $d \in \mathrm{dom}(\theta_{\text{cap}})$ **do**
  $L \leftarrow \sum_{b' : X(b') \in \mathcal{S}_d} e(b')$ `// current day-`$d$ `enrollment`
  **foreach** *unassigned* $b \in \mathcal{B}$*, in non-decreasing* $e(b)$ *order* **do**
    **if** $F \cap \mathcal{S}_d = \emptyset$ **or** $L + e(b) > \theta_{\text{cap}}(d)$ **then**
      **break**
    **end**
    $s \leftarrow \min(F \cap \mathcal{S}_d)$; $X(b) \leftarrow s$; $F \leftarrow F \setminus \{s\}$; $L \leftarrow L + e(b)$
  **end**
**end**

`// Stage 4: default placement`

**foreach** *unassigned* $b \in \mathcal{B}$ **do**
  **if** $X^0(b) \in F$ **then**
    $X(b) \leftarrow X^0(b)$
  **else**
    $X(b) \leftarrow \min F$
  **end**
  $F \leftarrow F \setminus \{X(b)\}$
**end**

**return** $X$


Bruno, G., Cavola, M., Diglio, A., Laporte, G., and Piccolo, C. (2021). Reorganizing postal collection operations in urban areas as a result of declining mail volumes–a case study in Bologna. *Journal of the Operational Research Society*, 72(7):1591–1606.

Chen, H., Constante-Flores, G. E., Mantri, K. S. I., Kompalli, S. M., Ahluwalia, A. S., and Li, C. (2025a). OptiChat: Bridging optimization models and practitioners with large language models. *INFORMS Journal on Data Science*.

Chen, L. and Miller-Hooks, E. (2012). Resilience: an indicator of recovery capability in intermodal

freight transport. *Transportation Science*, 46(1):109–123.

Chen, Z., Zhang, X., Zope, H., Barbalho, H., Mellou, K., Molinaro, M., Kulkarni, J., Menache, I., and Li, S. (2025b). Optimind: Teaching llms to think like optimization experts. *arXiv preprint arXiv:2509.22979*.

Doerr, B., Doerr, C., and Neumann, F. (2019). Fast re-optimization via structural diversity. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '19, pages 233–241.

Dong, Y., Maravelias, C. T., and Jerome, N. F. (2018). Reoptimization framework and policy analysis for maritime inventory routing under uncertainty. *Optimization and Engineering*, 19:937–976.

Drossman, J., Jacquillat, A., and Martin, S. (2026). Let's have a conversation: Designing and evaluating LLM agents for interactive optimization. *arXiv preprint arXiv:2604.02666*.

D'Ariano, A., Albrecht, T., Allan, J., Brebbia, C., and Rumsey, A. (2010). Running time re-optimization during real-time timetable perturbations. *Timetable Planning and Information Quality*, 1:147–156.

Er Raqabi, E. M., Beljadid, A., Bennouna, M. A., Bennouna, R., Boussaadi, L., El Hachemi, N., El Hallaoui, I., Fender, M., Jamali, M. A., Si Hammou, N., et al. (2025). OCP optimizes its supply chain for africa. *INFORMS Journal on Applied Analytics*.

Er Raqabi, E. M., Himmich, I., El Hachemi, N., El Hallaoui, I., and Soumis, F. (2023). Incremental lns framework for integrated production, inventory, and vessel scheduling: Application to a global supply chain. *Omega*, 116:102821.

Er Raqabi, E. M., Wu, Y., El Hallaoui, I., Soumis, F., et al. (2024). Towards resilience: Primal large-scale re-optimization. *Transportation Research Part E: Logistics and Transportation Review*, 192:103819.

Gauthier, P. (2024). Aider: AI pair programming in your terminal. https://aider.chat/. Accessed 2026.

Hasani, A., Mokhtari, H., and Fattahi, M. (2021). A multi-objective optimization approach for green and resilient supply chain network design: A real-life case study. *Journal of Cleaner Production*, 278:123199.

Hassani, R., Desaulniers, G., and Elhallaoui, I. (2020). Real-time personnel re-scheduling after a minor disruption in the retail industry. *Computers & Operations Research*, 120:104952.

Huang, C., Tang, Z., Hu, S., Jiang, R., Zheng, X., Ge, D., Wang, B., and Wang, Z. (2025). Orlm: A customizable framework in training large models for automated optimization modeling. *Operations Research*.

Jovine, A. S., Ye, T., Bahk, F., Wang, J., Ford, M., Shmoys, D. B., and Frazier, P. I. (2026). Listen to your preferences: An llm framework for multi-objective selection. In *Proceedings of the 35th International Joint Conference on Artificial Intelligence (IJCAI-ECAI 2026)*.

Klamkin, M., Deza, A., Cheng, S., Zhao, H., and Van Hentenryck, P. (2025). Dualschool: How reliable are llms for optimization education? *arXiv preprint arXiv:2505.21775*.

Kong, M., Qu, A., Guo, X., Ouyang, W., Jiang, C., Zheng, H., Ma, Y., Zhuang, D., Tang, Y., Li, J., et al. (2025). AlphaOPT: Formulating optimization programs with self-improving llm experience library. *arXiv preprint arXiv:2510.18428*.

Lawless, C., Li, Y., Wikum, A., Udell, M., and Vitercik, E. (2025). LLMs for cold-start cutting plane separator configuration. In *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pages 51–69. Springer.

Lawless, C., Schoeffer, J., Le, L., Rowan, K., Sen, S., St. Hill, C., Suh, J., and Sarrafzadeh, B. (2024). "I want it that way": Enabling interactive decision support using large language models and constraint programming. *ACM Transactions on Interactive Intelligent Systems*, 14(3):1–33.

Li, B., Mellou, K., Zhang, B., Pathuri, J., and Menache, I. (2023). Large language models for supply chain optimization. *arXiv preprint arXiv:2307.03875*.

Liang, K., Lu, Y., Mao, J., Sun, S., Zeng, C., Jin, X., Qin, H., Zhu, R., Teo, C.-P., et al. (2025). LLM for large-scale optimization model auto-formulation: A lightweight few-shot learning approach. *Congcong and Jin, Xiao and Qin, Hanzhang and Zhu, Ruihao and Teo, Chung-Piaw, LLM for Large-Scale Optimization Model Auto-Formulation: A Lightweight Few-Shot Learning Approach (June 28, 2025)*.

Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J., Mehrabian, A., et al. (2025). Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*.

Okeukwu-Ogbonnaya, A., Amatapu, R., Bergtold, J., and Amariucai, G. (2025). Llm-based community surveys for operational decision making in interconnected utility infrastructures. In *International Conference on Advances in Social Networks Analysis and Mining*, pages 264–274. Springer.

OpenAI (2026). OpenAI API Platform. https://platform.openai.com/. Accessed 2026.

Ramamonjison, R., Yu, T., Li, R., Li, H., Carenini, G., Ghaddar, B., He, S., Mostajabdaveh, M., Banitalebi-Dehkordi, A., Zhou, Z., et al. (2023). Nl4opt competition: Formulating optimization problems based on their natural language descriptions. In *NeurIPS 2022 competition track*, pages 189–203. PMLR.

Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J., Ellenberg, J. S., Wang, P., Fawzi, O., et al. (2024). Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475.

Schieber, B., Shachnai, H., Tamir, G., and Tamir, T. (2018). A theory and algorithms for combinatorial reoptimization. *Algorithmica*, 80:576–607.

Shu, X., Qian, H., Lu, X., Zhou, J., Zhou, A., Yu, Y., et al. (2025). LLMOPT: Learning to define and solve general optimization problems from scratch. In *International Conference on Learning Representations*, volume 2025, pages 101580–101606.

Simchi-Levi, D., Dai, T., Menache, I., and Wu, M. X. (2025a). Democratizing optimization with generative AI. *Available at SSRN 5511218*.

Simchi-Levi, D., Mellou, K., Menache, I., and Pathuri, J. (2025b). Large language models for supply chain decisions. *arXiv preprint arXiv:2507.21502*.

Xiao, Z., Xie, J., Xu, L., Guan, S., Zhu, J., Han, X., Fu, X., Yu, W., Wu, H., Shi, W., et al. (2025). A survey of optimization modeling meets llms: Progress and future directions. *arXiv preprint arXiv:2508.10047*.

Xiao, Z., Zhang, D., Wu, Y., Xu, L., Wang, Y., Han, X., Fu, X., Zhong, T., Zeng, J., Song, M., et al. (2024). Chain-of-experts: When llms meet complex operations research problems. In *International Conference on Learning Representations*, volume 2024, pages 48519–48537.

Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q. V., Zhou, D., and Chen, X. (2024). Large language models as optimizers. In *International Conference on Learning Representations*, volume 2024, pages 12028–12068.

Ye, H., Wang, J., Cao, Z., Berto, F., Hua, C., Kim, H., Park, J., and Song, G. (2024). Reevo: Large language models as hyper-heuristics with reflective evolution. *Advances in neural information processing systems*, 37:43571–43608.

Ye, T., Jovine, A. S., Van Osselaer, W., Zhu, Q., and Shmoys, D. B. (2026). Cornell university uses integer programming to optimize final exam scheduling. *INFORMS Journal on Applied Analytics*, 56(2):159–177.

Zhang, H., Zhu, Q., and Dou, Z. (2025). Enhancing reranking for recommendation with llms through user preference retrieval. *Proceedings of the 31st International Conference on Computational Linguistics*, pages 658–671.

Zhang, Y., Long, Y., Ni, Y., Huang, L., Wang, X., and Liu, J. (2026). Hierarchical optimization via llm-guided objective evolution for mobility-on-demand systems. *Advances in Neural Information Processing Systems*, 38:149894–149933.

Zheng, Z., Xie, Z., Wang, Z., and Hooi, B. (2025). Monte carlo tree search for comprehensive exploration in llm-based automatic heuristic design. *arXiv preprint arXiv:2501.08603*.

Zhou, C., Yang, J., Xin, L., Chen, Y., He, Z., and Ge, D. (2025). Auto-formulating dynamic programming problems with large language models. *arXiv preprint arXiv:2507.11737*.